Swarm, not my area.

D^2SLAM: Decentralized and Distributed Collaborative Visual-inertial SLAM System for Aerial Swarm

Hao Xu, Peize Liu, Xinyi Chen, Shaojie Shen

arXiv:2211.01538v4 [cs.RO] 23 Jun 2024

***Abstract*—Collaborative simultaneous localization and mapping (CSLAM) is essential for autonomous aerial swarms, laying the foundation for downstream algorithms such as planning and control. To address existing CSLAM systems' limitations in relative localization accuracy, crucial for close-range UAV collaboration, this paper introduces D^2SLAM—a novel decentralized and distributed CSLAM system. D^2SLAM innovatively manages near-field estimation for precise relative state estimation in proximity and far-field estimation for consistent global trajectories. Its adaptable front-end supports both stereo and omnidirectional cameras, catering to various operational needs and overcoming field-of-view challenges in aerial swarms. Experiments demonstrate D^2SLAM's effectiveness in accurate egomotion estimation, relative localization, and global consistency. Enhanced by distributed optimization algorithms, D^2SLAM exhibits remarkable scalability and resilience to network delays, making it well-suited for a wide range of real-world aerial swarm applications. The adaptability and proven performance of D^2SLAM represent a significant advancement in autonomous aerial swarm technology.**

***Index Terms*—Aerial systems: perception and autonomy, multi-robot systems, SLAM, swarms.**

I. Introduction

LOCALIZATION technology is critical for highly autonomous robot swarms. Unlike individual mobile robots, swarm robots are required to estimate not only their own state but also the states of other robots in the swarm. In recent years, simultaneous localization and mapping technology (SLAM) [1]–[4] for swarm robots, including aerial swarms [5], [6], has been greatly developed. These methods are well-known as collaborative SLAM (CSLAM) or multi-robot SLAM.

Considering the size, weight, and power (SWaP) constraints of aerial robot platforms, the primary goal of most SLAM algorithms on aerial robots is to provide input (such as trajectories and ESDF map) for planning and control, as shown in Fig. 1, rather than producing high-quality mesh. Let's explore a few categories of practical tasks of aerial swarms to summarize the technology requirements of CSLAM. The first category is self-assemble aerial swarm [7] and cooperative transportation [8] using multiple UAVs, in this case, UAVs in the swarm require very precise relative localization (centimeter level) to cooperate with each other at a very close distance (usually less than a meter). The second category is inter-UAV collision avoidance [9] and formation flight [10], in this case, high accuracy relative localization (centimeter-level to decimeter-level) at a few meters' distance is required to avoid collision and maintain flight formation. Another typical task of aerial swarm is cooperative exploration of unknown space. In some typical task, e.g., unknown space exploration [6], it's essential to have a high accuracy relative localization when UAVs are near to each other to avoid inter-UAV collision for this task. However, when UAVs start to explore the unknown space and fly far away from each other, the relative localization accuracy is not important, and the global consistency of the estimated trajectories is more important to build up the global map of the unknown space.

In summary, for close-range operations, aerial swarms require high-precision relative localization. At longer distances, however, this precision becomes less critical, with a greater

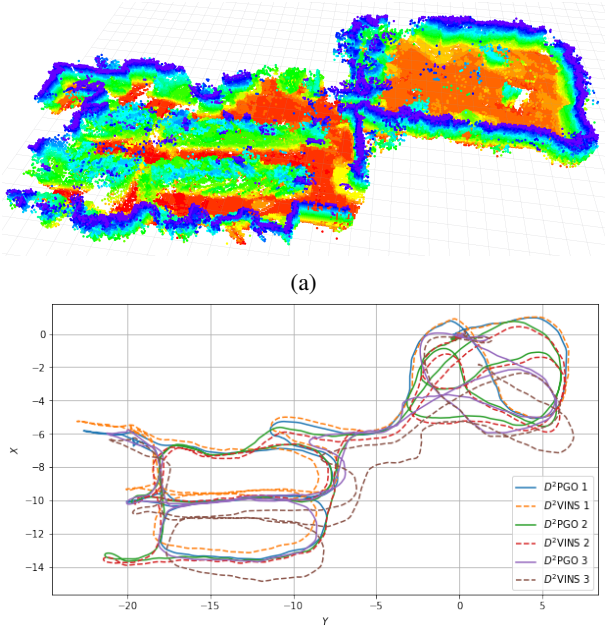


Fig. 1: The demonstration of D^2SLAM in the HKUST RI dataset: a) The dense map generated by D^2SLAM using TSDF reconstruction, showing only the surface voxels from TSDF. b) The estimated trajectories of D^2VINS (for near-field state estimation) and D^2PGO (for far-field state estimation) in a three-UAV scenario.

Manuscript was accepted by IEEE Transactions on Robotics on May 13, 2024. This work was supported by the Research Grants Council General Research Fund (RGC GRF) project RMGS20EG20, and the HKUST-DJI Joint Innovation Laboratory. *(Corresponding author: Hao Xu.)*

All authors are with the Department of Electronic and Computer Engineering, Hong Kong University of Science and Technology, Hong Kong, China. {hxubc, pliuan, xchencq}@connect.ust.hk, eeshaojie@ust.hk

emphasis on the global consistency of state estimation. Additionally, accurate ego-motion estimation is vital for stable flight in all tasks.

CSLAM algorithms are influenced not only by task-specific requirements but also by communication constraints. Common communication modes in robot swarms typically encompass centralized systems such as WLAN, mainly used in laboratory settings, and wireless ad hoc networks. The latter, a decentralized approach, is particularly suitable for complex real-world scenarios where setting up a router for centralized communication is not feasible. In the latter approach, typically seen in high-mobility aerial swarms [1], robots form mesh networks with communication limited by factors like occlusion, distance, and interference. While these ad hoc networks provide effective communication at close ranges, they face challenges in bandwidth and stability over longer distances.

Moreover, the computational architecture of CSLAM may vary, primarily encompassing two major forms for robot swarms: centralized and decentralized. Centralized CSLAM processes information on a ground-station server, requiring stable network connectivity, while decentralized CSLAM [1], [3] adapts to a broader range of environments. A well-designed decentralized approach is less dependent on stable network connectivity. However, decentralized methods may only partially address the issue. Some approaches, such as [1], [11], are considered decentralized due to the lack of a central node, yet they replicate the same computations across multiple robots. As the number of UAVs in a swarm increases, these methods are likely to encounter computational bottlenecks. Compared to these approaches, distributed CSLAM offers the advantage of reduced (but not cost-free) computational overhead per UAV, and enhanced privacy, as only partial information needs to be shared [4]. The key advantage of distributed SLAM lies in its scalability, enabling applications in large-scale swarms.

Taking into account communication constraints and SWaP limitations, and driven by the notion that precise relative localization is essential only for nearby robots while global consistency gains importance as robots move farther apart, we propose D^2SLAM. This innovative system represents a novel approach to decentralized and distributed visual-inertial SLAM. Its state estimation combines two parts: near-field and far-field state estimation for aerial swarms. Near-field state estimation in D^2SLAM involves estimating high-precision real-time local state (e.g., visual-inertial odometry (VIO)) and relative state between UAVs when their onboard sensors have field-of-view (FoV) overlap and good communication. Far-field state estimation involves estimating trajectories with global consistency when UAVs are far apart or in non-line-of-sight, with limited communication. These two state estimation approaches effectively address the challenges associated with state estimation in aerial swarms, as previously outlined.

The near-field estimation module in D^2SLAM, named D^2VINS (decentralized and distributed visual-inertial navigation system), is a collaborative multi-robot VIO system using sparse features. Similar to standard single-robot VIO systems like VINS-Mono [12], D^2VINS maintains a local map using a sliding window and employs graph-based optimization for state estimation. Recent studies have shown significant advancements in applying the alternating direction method of multipliers (ADMM) [13], [14] to distributed bundle adjustment [14], [15]. Inspired by these developments, we have adopted ADMM in D^2VINS to effectively address distributed VIO challenges.

Given visual SLAM limitations, D^2VINS requires overlapping sensor FoVs for accurate real-time relative state estimation at close range. Omnidirectional vision systems [1], especially with our enhanced omnidirectional frontend, effectively address these issues in aerial swarms. With omnidirectional cameras, D^2VINS is not constrained by the UAVs' yaw angles, enabling precise relative localization when UAVs maintain line-of-sight, share environmental features, and communicate reliably. Commercial stereo cameras like the Intel Realsense D435i [1], widely used in advanced aerial swarms [5], [6], are also supported by D^2SLAM. Stereo cameras, requiring less computational power, are well-suited for UAVs with limited resources. However, their optimal usage, particularly adjusting UAV yaw for FoV overlap, falls beyond the scope of this paper.

The core of far-field estimation is D^2PGO (decentralized and distributed pose graph optimization). D^2PGO facilitates both relative and global localization through pose graph optimization, particularly when UAVs are distant or out of each other's line of sight. Unlike D^2VINS, which requires good communication conditions, D^2PGO functions effectively even in poor communication scenarios, characterized by unstable network latency and low bandwidth. Peng et al.'s distributed optimization method, ARock [16], adeptly handles network latency and asynchronous updates, effectively countering the network instability and delays in pose graph optimization. Thus, we introduce an ARock-based asynchronous distributed pose graph optimization algorithm in Sect. VII.

The main contributions of this paper are as follows:

- Introduction of D^2SLAM, a novel decentralized and distributed SLAM system, capable of achieving high-accuracy ego-motion and relative state estimation for nearby UAVs, as well as globally consistent trajectory estimation for distant or non-visible UAVs.
- We introduce D^2VINS, a distributed visual-inertial state estimator for multi-robot systems utilizing the ADMM approach. This system represents the first instance of a tightly-coupled visual-inertial odometry estimator based on distributed optimization, providing both ego-motion estimation and high-accuracy relative localization.
- D^2PGO, an ARock-based, asynchronous, distributed pose graph optimization algorithm, specifically designed for multiple robots is introduced. The adaptability of D^2PGO to communication delays and its proficiency in managing nonlinear issues make it an ideal solution for robot swarms. This is also the first attempt of using ARock in solving pose graph problems.
- We conducte extensive testing of D^2SLAM on an aerial swarming platform, encompassing both dataset evaluations and real-world experiments. Additionally, we have open-sourced the code and custom datasets[2].

[1]https://www.intelrealsense.com/depth-camera-d435i

[2]https://github.com/HKUST-Aerial-Robotics/D2SLAM

II. Related Works

A. Distributed SLAM Techniques

This subsection explores fundamental techniques in distributed SLAM, namely distributed pose graph optimization and distributed bundle adjustment:

1) Distributed pose graph optimization: Pose graph optimization (PGO) [17], [18], a technique derived from factor graph theory [19], is crucial in SLAM for re-localization [12], [20] and dense mapping [21]. When SLAM expanded to multi-robot collaboration, PGO was naturally incorporated [22]–[24]. Initial PGO applications in CSLAM utilized centralized servers to solve the pose graph problem [24], but faced scalability and communication challenges in swarm robot systems. Cunningham et al. proposed the first distributed SLAM method, DDF-SAM [22], using a constrained factor graph, a concept further refined in DDF-SAM2 [23]. Nevertheless, these methods maintain a neighborhood graph on each agent, leading to computational inefficiencies and scalability issues.

Significant advancements in distributed pose graph optimization (DPGO) began with Choudhary et al.'s distributed Gauss-Seidel (DGS) approach [4]. This method transforms the PGO problem into two linear problems, which are then distributedly solved using the Gauss-Seidel technique [25]. However, this method struggles with unfavorable convergence at high noise levels and necessitates synchronous operation. Later, the ASAPP method, based on distributed gradient descent, was proposed by Tian et al. [26], functioning as a distributed and asynchronous version of Riemann gradient descent [27]. This work was notable for being the first to incorporate communication considerations, introducing an asynchronous DPGO approach. Tian et al.'s subsequent work [28] introduced the DC2-PGO method using the Distributed Riemann-Staircase, a certifiably correct DPGO approach. These DPGO methods have been incorporated into various practical SLAM systems, such as DOOR-SLAM [3] that utilizes DGS, and Kimera-multi [2], [29] which employs both ASAPP and DC2-PGO.

2) Distributed bundle adjustment: Bundle adjustment (BA) [30], a core component in structure-from-motion (SfM) and sparse SLAM, has evolved with significant advancements in distributed processing. Eriksson et al. [15] pioneered a consensus-based distributed BA method, partitioning camera poses to enhance processing in a manner akin to the Alternating Direction Method of Multipliers (ADMM), though it requires extensive communication for landmark data. Addressing this challenge, Zhang et al. [14] developed a 'camera consensus' method, effectively reducing communication needs and improving convergence. [31] investigates a loosely coupled approach for fusing VIO with relative measurements in distributed VIO, utilizing ARock [16], an asynchronous distributed optimization algorithm. However, their focus on predominantly two-robot scenarios highlights the necessity for more versatile solutions. Additionally, the use of multi-state constraint Kalman filter (MSCKF) for distributed BA, as studied in [32]–[34], although promising, may suffer from accuracy loss due to the single linearization of measurements.

Building on these insights, our approach, inspired by [14], adopts a strategy of dividing landmarks into disjoint sets for an ADMM-based collaborative VIO in a distributed setting, aiming to enhance both accuracy and efficiency in multi-robot SLAM applications. Our method stands out as the first to tightly-coupled distributed visual-inertial odometry in contrast to existing ADMM-based distributed bundle adjustment approaches. Unlike the structure-from-motion methods [14], [15], which don't prioritize real-time performance, our method is designed for real-time operation. Compared to approach [31], our method is applicable to multi-robot scenarios, not just limited to two UAVs. A unique feature of D^2SLAM is the directly and tightly integration of both visual and IMU measurements, a capability absent in previous works. This enhancement maximizes the utility of visual-inertial inputs to improve accuracy, making our method highly suitable for aerial swarm applications.

B. Current CSLAM Systems

In this section, we review existing CSLAM systems, highlighting their strengths and limitations. DPGO based systems, such as [2], [3], are known for their global consistency and distributed capabilities, yet they often lack in high relative localization accuracy. Systems employing BA on landmarks, including [31]–[34], offer multi-camera state estimation but typically fall short in global consistency, which limits their broader application.

Additionally, methods incorporating relative measurements like UWB ranging or visual detection are gaining attention. For example, [11], [35], [36] apply UWB-odometry fusion for relative state estimation but face challenges in achieving global consistency. [36] uses ARock for backend optimization to scale up UWB-odometry fusion, while [37] integrates UWB anchors for enhanced global consistency, albeit with reliance on ground infrastructure. On the other hand, [1] utilizes a map-based localization approach for achieving global consistency, though UWB systems often encounter issues like obscuration and radio interference in real-world environments.

Furthermore, visual-detection-based approaches such as [1], [11], [38] face limitations due to reliance on pre-trained models, particularly when UAVs are at a distance.

Contrastingly, D^2SLAM not only ensures global consistency but also high relative localization accuracy at close ranges. Its distributed architecture is particularly suited for scalable swarm applications. Diverging from approaches like [2], [5], [9], [32], [33], [35], [36] that depend on ground truth poses for initialization, D^2SLAM enhances system flexibility by allowing runtime merging of different robots' reference systems and sparse maps.

Unlike the distributed algorithms typically limited to dataset validation or simulation [2], [14], [15], [32], [33], our approach has undergone extensive testing both through in-the-loop flight experiments. D^2SLAM demonstrates promising performance in robotic systems' perception-control loops, showcasing its practical usability and effectiveness.

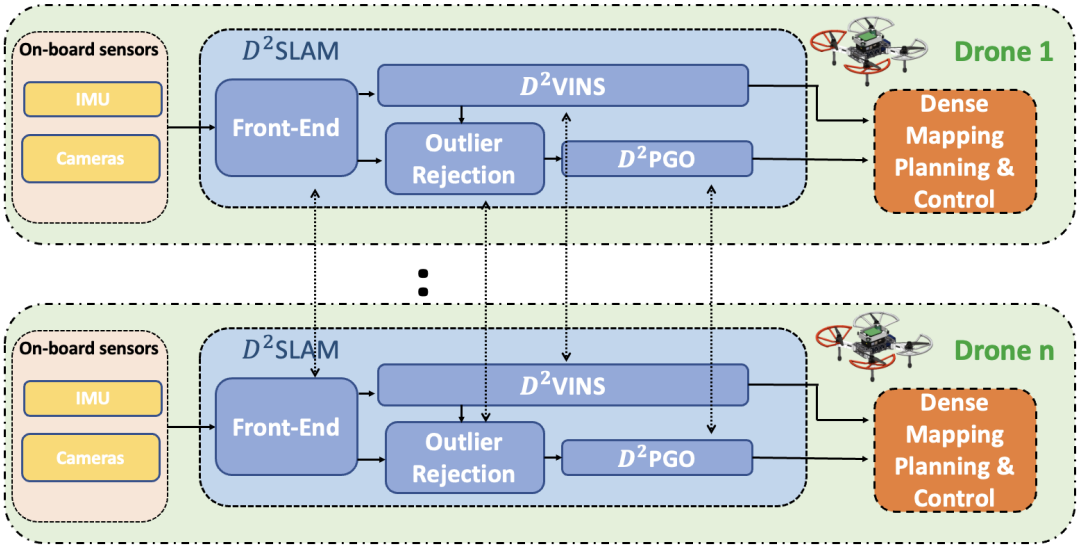


Fig. 2: The architecture of D^2SLAM. The D^2SLAM is independently running on each UAV. The data will be first processed by front-end and then sent to back-end for state estimation. Results can be utilized for dense mapping, planning and control.

III. Preliminary

This paper adopts the symbol system from [1], details of which are not repeated here. We consider an aerial swarm consisting of N UAVs, each with a unique ID, $i \in \mathcal{D}$, where $\mathcal{D} = \{1, 2, \cdots, N\}$ represents the set of all UAVs in the swarm. By default, the swarm's global reference system is established based on the initial position of UAV k, unless stated otherwise. D^2SLAM operates independently on each UAV, with a focus on its implementation on the i-th UAV unless specified. Details on establishing a unified reference system among the UAVs will be provided in Sect. IV-C.

A. State Estimation Problem of CSLAM on Aerial Swarm

The state estimation problem in an aerial swarm is defined as follows: for each UAV $k \in \mathcal{D}$, estimate the 6-degree-of-freedom (DoF) pose ${}^{v_k}\mathbf{T}_i^t$ of every other UAV $i \in \mathcal{D}_a^k$ at time t in UAV k's local frame, where $\mathcal{D}_a^k$ represents the set of UAVs in communication with UAV k. This problem is divided into two key parts for UAV k:

1) Estimation of UAV k's own ego-motion state $\hat{\mathbf{T}}_k^t$ in its local frame.
2) Estimation of the state ${}^{v_k}\hat{\mathbf{T}}_i^t$ of any other UAV i.

In SLAM research [1], [39]–[41], global consistency is understood as the ability to bound the absolute trajectory error (ATE) [42], ensuring that state estimation errors do not progressively increase with the robot's movement. In this paper, it means that the estimated poses ${}^{v_k}\hat{\mathbf{T}}_i^t$ remain accurate and do not suffer from drift as the robot moves.

B. Decentralized vs Distributed

Decentralized and distributed systems, often used interchangeably, have distinct meanings. Decentralized systems operate without a central node, with algorithms running independently across multiple nodes. In contrast, distributed systems, as defined in [44], spread computation or resources like storage across multiple nodes, aiming for non-redundant processing. However, decentralized systems can involve significant redundant computation, while distributed systems may still rely on a central node for coordination. The relationship between decentralization and distribution is illustrated in Fig. 3. For instance, Omni-swarm [11] is decentralized but not distributed, lacking a central node but involving redundant optimization across the swarm, leading to computational inefficiency. Conversely, Asynchronous Distributed ADMM [43] employs a master-worker structure that distributes computation across multiple nodes, yet relies on a central node.

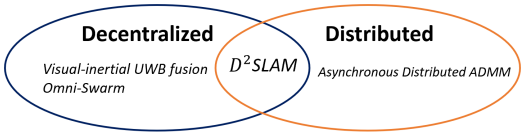


Fig. 3: Visual-inertial UWB fusion [11] and Omni-Swarm [1] are decentralized, and asynchronous Distributed ADMM [43] is distributed. D^2SLAM is both decentralized and distributed.

This paper aims to develop a fully distributed and decentralized (D^2) CSLAM algorithm, D^2SLAM. This approach ensures robustness against single node failures and communication losses, enabling the swarm to dynamically form subswarms or integrate into a larger swarm. Moreover, D^2SLAM efficiently utilizes the computational resources of each UAV, enhancing overall system performance.

IV. System Overview

A. D^2SLAM System Architecture

Fig. 2 depicts the architecture of D^2SLAM. On each UAV, the front-end processes visual inputs for key frame extraction, sparse feature tracking, and loop closure detection. These processed data are then forwarded to the back-end for two types of state estimations: collaborative visual odometry (near-field state estimation) and pose graph optimization (far-field state estimation), handled by D^2VINS and D^2PGO, respectively.

The resulting estimations support further processes like dense map building, planning, and control.

B. Communication Modes

D^2SLAM employs a combination of three distinct communication modes to enhance system robustness and minimize communication overhead:

- **Discover mode**: Activated when UAVs' relative states are unknown, this mode involves broadcasting complete keyframes (Sect. V-A) and states for both near (Sect. VI) and far-field (Sect. VII) state estimation. This facilitates rapid initialization of relative state estimation and supports map merging (Sect. IV-C) for integrating UAVs' reference systems.
- **Near mode**: When other UAVs are predicted to be in proximity (within 3 to 5 meters), based on current state estimates, D^2SLAM broadcasts complete keyframes and states necessary for near and far-field state estimation. The proximity prediction relies on state estimation from both D^2PGO and D^2VINS.
- **Far mode**: Used when UAVs are distant from each other, this mode transmits compact keyframes (Sect. V-A) and performs distributed loop closure detection (Sect. V-E). It broadcasts states for far-field state estimation. In far mode, near-field state estimation simplifies to single-robot VIO.

Fig. 4a depicts D^2SLAM's communication mode transitions. After initialization, near-field state estimation is limited to near mode to save bandwidth, while far-field state estimation operates in all modes for global consistency. Details on data transmission per mode are provided in later sections.

C. Multi-Robot Map Merging

In D^2SLAM, each robot starts with its own local reference frame $\mathbf{F}_i$ at boot time. When map-merging occurs, typically during encounters with other UAVs or revisiting areas covered by others, the UAVs align their coordinate systems to form a unified reference system. This alignment, triggered by relative measurements, integrates the coordinate systems without exchanging landmarks, allowing each UAV to maintain its individual sparse map. Specifically, the system adopts the reference frame of the UAV with the smaller ID, and the larger ID UAV's states in D^2VINS and D^2PGO are converted to this unified system.

Fig. 4b illustrates the map-merging process in D^2SLAM. UAVs continuously transmit heartbeat packets for detection by others, facilitating their discovery. During the discover mode, D^2SLAM broadcasts information until the swarm's UAVs are unified under a single coordinate system. After completing map-merging, subsequent state estimations proceed without this process, unless new UAVs join the swarm.

D. Operating Conditions of D^2SLAM

D^2SLAM adapts to environmental and communication constraints by transitioning to single-robot SLAM or VIO when necessary. This ensures aerial swarm stability and enables short-term formation flight using only VIO, as shown in [10]. Detailed discussions on these limitations follow.

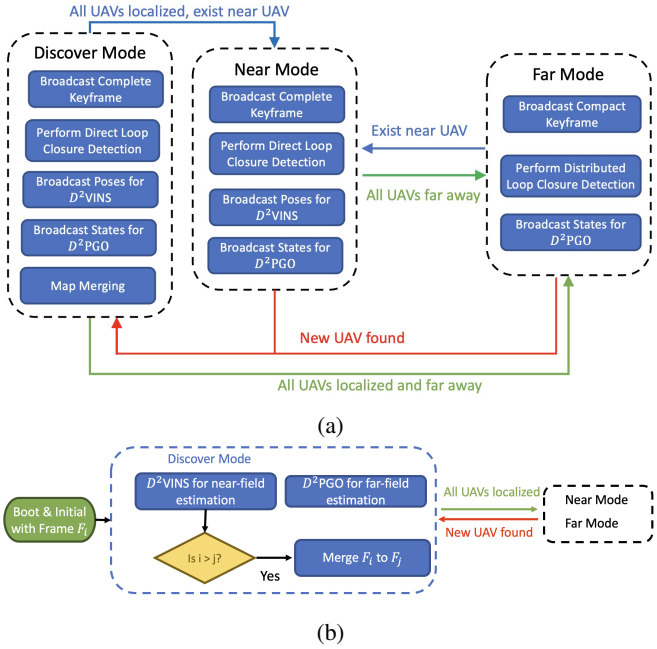


Fig. 4: a) A state machine governing D^2SLAM's communication modes begins in discover mode. In this initial phase, each UAV broadcasts its complete keyframe to support rapid initialization and transmits data essential for D^2VINS. Once UAVs have successfully initialized their relative states, D^2SLAM transitions to either near or far mode, depending on their estimated relative positions. The system can fluidly alternate between near and far modes in response to changes in UAV positioning. Importantly, if a new UAV enters the system, typically seen during the initialization phase, D^2SLAM reverts to discover mode to incorporate this new member. b) Map merging occurs during the discover mode, as shown in the flow charts. If near or far-field state estimation identifies other UAVs, the UAV with the higher index transfers its map state to the UAV with the lower index.

1) Communication: In D^2SLAM, a loss of communication triggers a transition to single-robot SLAM, referred to as the near communication mode. Near-field state estimation requires extensive data exchange and real-time performance, hence a robust, low-latency network is essential. Conversely, far-field state estimation can function effectively with lower bandwidth and some latency.

2) Environments: In open environments with limited environmental features, such as grasslands or rough walls, sparse visual SLAM faces challenges in feature matching for relative localization and loop closure detection. In response to these limitations, our system is designed to downgrade to single-robot VIO, ensuring flight safety under such conditions.

3) Field of View: Accurate relative localization in D^2SLAM is contingent on FoV overlap and a sufficient number of common features. Yaw alignment is crucial for FoV-limited cameras, but this requirement is eliminated with omnidirectional cameras. The local precision of D^2VINS helps offset brief losses of common features, ensuring reliable relative state estimation. When UAVs lose shared vision with others for an extended period, D^2VINS degrades to single-robot VIO. We believe that the need for accurate relative localization lessens when UAVs are either out of sight for extended periods or too distant to share features.

V. Front-end

D^2SLAM features a versatile and efficient front-end that supports various camera configurations, including omnidirectional and stereo cameras.

Structure of front-end: Fig. 5 illustrates the components of our front-end, which include: 1) Pre-processing fisheye images from omnidirectional cameras; 2) General pre-processing for extracting landmarks and global descriptors; 3) Feature tracking for the local UAV; 4) Sparse feature matching across multiple UAVs; 5) Distributed loop closure detection.

Landmarks & features: Similar to other sparse visual SLAM systems, D^2SLAM utilizes landmark points to model the external environment. It treats a set of consistently tracked or matched 2D feature points across images as measurements of identical landmarks. Subsequent sections will detail the methods for tracking and matching landmark-related features, both by individual UAVs and collaboratively among multiple UAVs.

A. Visual Data Preprocessing

In this work, as depicted in Fig. 7, we employ a quad fisheye camera system as an omnidirectional camera. This configuration, compared to dual fisheye systems [1], [45], offers more effective FoV usage and generates superior quality depth information. Its advantages have led to its adoption in both SLAM research [46] and advanced commercial UAV products, such as the DJI Mavic 3 [3].

Fisheye cameras produce significantly distorted images, challenging direct application of conventional vision techniques, as shown in Fig. 6a. Following [47], we first reproject these images into a cylindrical projection (see Fig. 6b) before processing. [47] demonstrates that applying CNNs trained on public datasets to these reprojected images yields effective results.

In D^2SLAM, to conserve communication bandwidth, complete keyframe information, such as SuperPoint features and global descriptors for each camera view, is broadcasted to other UAVs only in discover or near communication modes. On the other hand, the compact keyframes, include only the NetVLAD descriptors, are used in far mode to save bandwidth.

B. Descriptor Extraction & Dimension Reduction

We employ MobileNetVLAD [48] to construct a global descriptor for each camera view, and SuperPoint [49] for landmark extraction, coupled with PCA for dimension reduction of the descriptors. These techniques, validated in our previous work [1], are not elaborated here for brevity.

C. Sparse Feature Tracking

Sparse visual SLAM typically employs two types of feature tracking: 1) Lucas-Kanade (LK) method, as in VINS-Mono [12], offering robustness in feature-poor environments like grass, and 2) Descriptor-based matching, notably in ORB-SLAM [50], with CNN-based approaches excelling in handling large parallax for multi-UAV matching and loop closure detection, though less effective in feature-sparse areas. Given the importance of both multi-UAV feature matching and robustness in our work, we adopt a hybrid front-end to meet these requirements, illustrated in Fig. 6b.

Each camera view in D^2SLAM tracks up to N_{max} (typically 100-200) landmarks. Initially, we extract $N_{max_{sp}}$ (50-150) SuperPoint [49] sparse features and their descriptors, assuming N_{sp} successful extractions. SuperPoint is preferred for its robustness over ORB features [50]. These landmarks are tracked inter-frame using a kNN matcher with ratio test [51]. Additionally, Shi-Tomasi corners are detected to fill the remaining quota up to N_{max} and tracked with the LK method. SuperPoint features are utilized for loop closure detection and multi-UAV matching, while LK landmarks aid ego-motion estimation. In feature-scarce environments, if no SuperPoint features are detected ($N_{sp} = 0$), the system defaults to single-robot VIO. Similiar to [12], keyframe determination is based on parallax and the count of new landmarks after feature tracking.

Upon completing per-frame tracking, multi-view feature tracking is applied across different cameras on the same UAV. For stereo inputs, matching SuperPoint features and LK optical flow between left and right cameras is straightforward. In contrast, with quad fisheye cameras, as illustrated in Fig. 7, each camera shares landmarks only within the half-plane of its neighboring camera. Therefore, feature matching for SuperPoint features is limited to these corresponding half-planes. Landmarks' positions are predicted based on the cameras' internal and external parameters, serving as initial values for solving the LK optical flow.

D. Multi-UAV Sparse Feature Matching

When a new keyframe is received, the system first searches the sliding window for the most recently matching keyframe, determined by their global descriptors' inner product surpassing the threshold τ_{mg} (default 0.8). Following a successful match, kNN feature matching with ratio test is performed on the Superpoint features of these keyframes to establish feature correspondences. Subsequently, once a match is confirmed, we combine the corresponding landmark points of each matched feature pair into an inter-UAV landmark.

E. Distributed Loop Closure Detection

Following [1], we utilize Faiss [52] for whole image searching, kNN for feature matching, Perspective-n-Point (PnP) for relative pose extraction, and implement outlier rejection. Subsequently, loop edges are processed by the backend for pose graph optimization. As illustrated in Fig. 8, D^2SLAM adopts distributed loop closure detection [2], [3] to minimize communication bandwidth in far mode, and direct loop clousure detection [1] in near and discover mode.

Unlike [1], in D^2SLAM, we use 3D positions of Superpoint landmarks, estimated by D^2VINS from historical frames, along with 2D observations from the current frame for PnP computation. Additionally, with quad fisheye cameras as input, solving a multi-camera PnP problem becomes necessary for relative pose extraction and geometric verification. In practice,

[3]https://www.dji.com/mavic-3

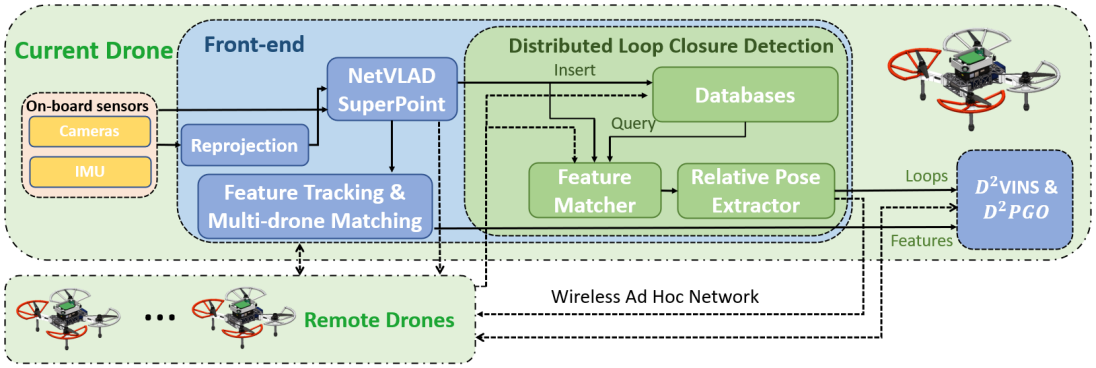
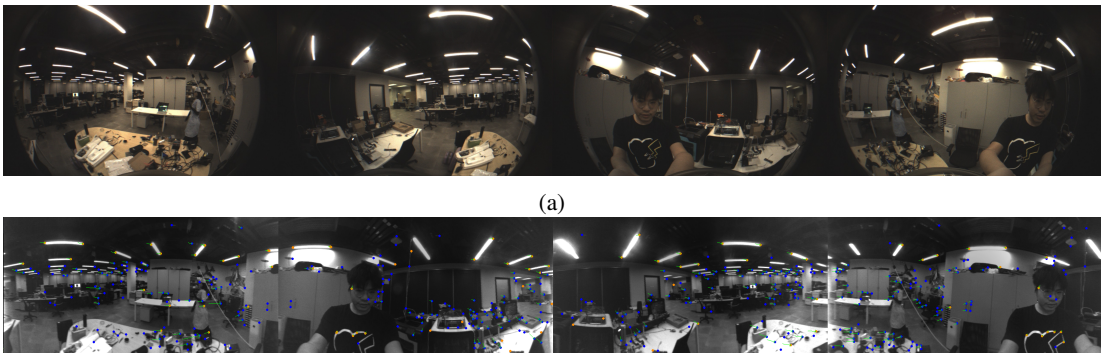


Fig. 5: The front-end of D^2SLAM processes visual data in several stages. Initially, the data undergoes reprojection, followed by extraction of global descriptors and features. Subsequently, it is used for feature tracking, multi-UAV feature matching, and loop closure detection. The final results are then fused in the back-end.



(a)

(b)

Fig. 6: Fisheye Image Demonstration: (a) Original fisheye images; (b) Reprojected fisheye images featuring tracking. Blue points represent SuperPoint features, while orange points indicate LK features, and green arrows depict the inter-frame movements of these features.

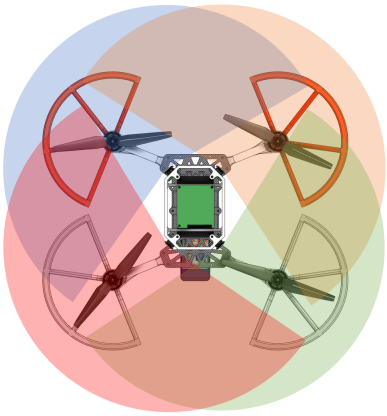


Fig. 7: Omnidirectional Camera Configuration: The transparent areas represent the Field of View (FoV) of each camera. These cameras boast a lateral FoV of approximately 200 degrees and feature an overlap of 110 degrees between adjacent cameras.

we initially apply the UPnP RANSAC algorithm [53] for pose estimation and outlier rejection. Subsequent inlier results are then used in a bundle adjustment problem for multi-camera PnP [54], enhancing pose accuracy.

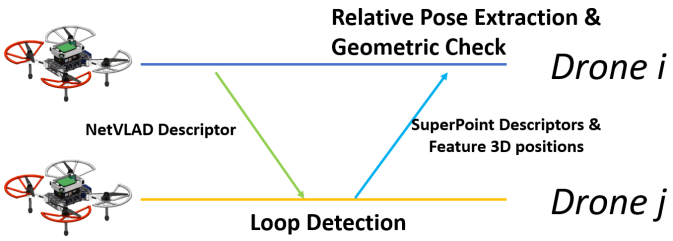


Fig. 8: Demonstration of distributed loop closure detection: UAVs transmit compact keyframes, which include NetVLAD global descriptors, across the swarm. Upon receiving this data from UAV i, UAV j initiates loop closure detection. In case of a successful match, UAV j sends a complete keyframe containing landmark information back to UAV i, enabling the extraction of the final relative pose.

VI. Near-field state estimation: D^2VINS

A. Problem Formulation

To facilitate near-field state estimation in D^2SLAM, we introduce D^2VINS, a collaborative visual-inertial odometry. Designed for real-time, accurate 6-DoF odometry estimation of both the local UAV and nearby UAVs, D^2VINS operates on a graph optimization framework with a sliding window, akin to [12]. Its state estimation problem is modeled using a factor graph [19], depicted in Fig. 9. State estimation within the sliding window is achieved by maximizing the a posteriori (MAP) inference of this factor graph through distributed non-

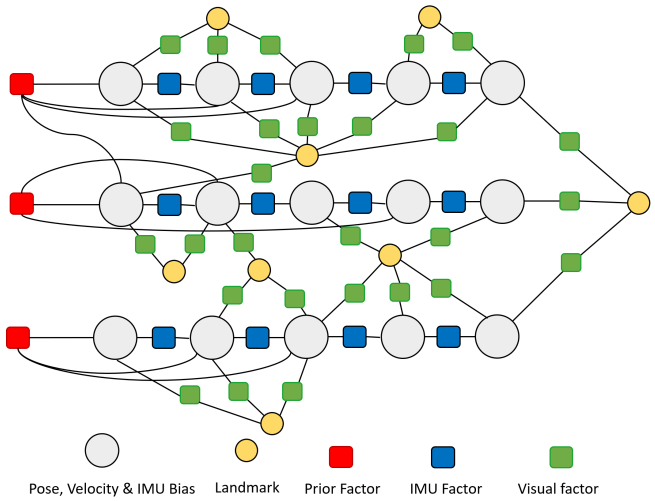


Fig. 9: A demonstration of the factor graph in D^2VINS. The states within the sliding window, comprising pose, velocity, and IMU bias, are interconnected by various factors including IMU, visual, and prior factors.

linear least squares optimization. The state of the collaborative visual-inertial-odometry problem is defined as:

$$\begin{aligned}\mathcal{X} = \Big[& {}^{v_k}\tilde{\mathbf{T}}_0^{t_0}, {}^{v_k}\tilde{\mathbf{T}}_0^{t_1}, \cdots {}^{v_k}\tilde{\mathbf{T}}_0^{t_{m-1}}, \mathbf{b}_0^{t_0}, \mathbf{b}_0^{t_1}, \cdots \mathbf{b}_0^{t_{m-1}}, \\ & \cdots {}^{v_k}\tilde{\mathbf{T}}_{n-1}^{t_0}, {}^{v_k}\tilde{\mathbf{T}}_{n-1}^{t_1}, \cdots {}^{v_k}\tilde{\mathbf{T}}_{n-1}^{t_{m-1}}, \mathbf{b}_{n-1}^{t_0}, \mathbf{b}_{n-1}^{t_1}, \cdots \\ & \mathbf{b}_{n-1}^{t_{m-1}}, l_0, l_1, l_2, \cdots l_L \Big]^T,\end{aligned} \tag{1}$$

where ${}^{v_k}\tilde{\mathbf{T}}_i^{t_j}$ is the estimated pose of UAV i at time t_j in the local frame of UAV k, m is the length of the sliding window and n is the number of the UAVs in the swarm, $\mathbf{b}_i^{t_j} = [{}^{v_k}\mathbf{v}_i^{t_j}, b_a^{t_j}, b_g^{t_j}]$ is the velocity, acceleration bias and angular velocity bias of the IMU of UAV i at time t_j, L is the total number of the landmark. In this paper, we use the inverse depth parametrization for landmarks [55], where l_j represents the inverse of the distance from landmark j to the keyframe it is associated with. For convenience, we denote the set of all poses in $\mathcal{X}$ as $\mathcal{T}$. The *optimization problem for collaborative visual-inertial-odometry* is formulated as,

$$\begin{aligned}\min_{\mathcal{X}} \Bigg\{ & \|\mathbf{r}_p - \mathbf{H}_p\mathcal{X}\|^2 + \sum_{k\in\mathcal{B}} \left\| r_{\mathcal{B}}(\tilde{z}_{b_k}^{b_{k+1}}, \mathcal{X}) \right\|_{\Sigma_{\mathcal{B}}} + \\ & \sum_{k\in\mathcal{L}} \rho\left(\|r_{\mathcal{L}}(\tilde{z}_{l_k}, \mathcal{X})\|_{\Sigma_{\mathcal{L}}} \right) \Bigg\}, \\ & s.t.\ \mathbf{P}_i \in \mathbf{SE}(3),\ \forall \mathbf{P}_i \in \mathcal{T}\end{aligned} \tag{2}$$

where $\rho(\cdot)$ is the Huber norm [56], $\|(\cdot)\|_\Sigma$ is the Mahalanobis norm, $\mathcal{B}$ is the set of IMU factors, $\mathbf{r}_\mathcal{B}$ is the residual of the IMU measurement factor, $\mathcal{L}$ is the set of visual measurements, and $\mathbf{r}_\mathcal{L}$ is the visual measurement residual, $\mathbf{r}_\mathcal{B}$ and $\mathbf{r}_\mathcal{L}$ are define in [12]; $\|\mathbf{r}_p - \mathbf{H}_p\mathcal{X}\|^2$ is the prior factor generated from previous marginalization, which will be detailed in Sect. VI-D.

B. Decentralized and Distributed Optimization of Collaborative visual-inertial-odometry problem

1) Alternating Direction Method of Multipliers (ADMM) algorithm: In this section, the target of decentralized and distributed optimization is to solve the problem,

$$\begin{aligned}& \sum_{i=1}^{n} f_i(\mathbf{x}_i), \\ & s.t.\ \mathbf{x}_i = \mathbf{z},\end{aligned} \tag{3}$$

using decentralized (and distributed) ADMM method [13], [14], where $\mathbf{x}_i$ is the local state for UAV i and z is the global state, Problem (3) can be jointly solved by iteratively performing the following update on each UAV i,

$$\mathbf{x}_i^{t+1} = argmin_{\mathbf{x}_i}\left\{ f_i(\mathbf{x}_i) + \frac{\rho}{2}\|\mathbf{x}_i - (\mathbf{z}^t - \mathbf{u}_i^t)\|^2 \right\} \tag{4}$$

$$\mathbf{z}^{t+1} = \frac{1}{n}\sum_{i=1}^{N}\mathbf{x}_i^{t+1} \tag{5}$$

$$\mathbf{u}_i^{t+1} = \mathbf{u}_i^t + \mathbf{x}_i^{t+1} - \mathbf{z}^t, \tag{6}$$

Here, in each iteration, each UAV solves Problem (4) and updates its states by averaging the results obtained from all UAVs.

2) Problem splitting: As outlined in Sect. II-A2, effectively partitioning the problem and state is crucial in ADMM optimization. In D^2VINS, we split the landmarks $\mathcal{L}$ into disjoint sets $\mathcal{L}_1, \mathcal{L}_2, \cdots \mathcal{L}_N$, where $\mathcal{L}_i$ represents the set allocated to UAV i. This partitioning approach and the landmark distribution policy are further elaborated in Sect. VI-C.Additionally, we denote the set of poses observing landmarks in $\mathcal{L}_i$ as $\mathcal{T}_i$. Consequently, we define the local state $\mathcal{X}_i$ of UAV i as,

$$\mathcal{X}_i = \left[\mathcal{T}_i, \mathcal{L}_i, \mathbf{b}_i^{t_0}, \mathbf{b}_i^{t_1} \cdots \mathbf{b}_i^{t_{m-1}}\right]^T, \tag{7}$$

where $\mathbf{b}_i^{t_0}, \mathbf{b}_i^{t_1} \cdots \mathbf{b}_i^{t_{m-1}}$ are the state of the velocity and IMU bias of UAV i, they are only essential for local optimization. Due to the presence of the hybrid tracking feature method (Sect. V-C), $\mathcal{X}_i$ necessarily contains all i's pose in $\mathcal{X}$. This ensures that once all remote UAV are disconnected, D^2VINS can still estimate the ego-motion, i.e., degrade to a single-robot VIO.

3) Distributed optimization for D^2VINS: Rewriten Eq. (2) to the form of decentralized optimization (3), we have,

$$\begin{aligned}\min_{\mathcal{X}_i} & \sum_{i=1}^{N} f_{cvio_i}(\mathcal{X}_i) \\ & f_{cvio_i}(\mathcal{X}_i) = \|\mathbf{r}_{p_i} - \mathbf{H}_{p_i}\mathcal{X}_i\|^2 + \sum_{k\in\mathcal{B}} \left\| r_{\mathcal{B}_i}(\tilde{z}_{b_k}^{b_{k+1}}, \mathcal{X}_i) \right\|_{\Sigma_{\mathcal{B}_i}} \\ & + \sum_{k\in\mathcal{L}} \rho\left(\|r_{\mathcal{L}}(\tilde{z}_{l_k}, \mathcal{X}_i)\|_{\Sigma_{\mathcal{L}}} \right) \\ & s.t.\ \mathcal{X}_i = \mathbf{E}_i\mathcal{X}, \\ & s.t.\ \mathbf{P}_j \in \mathbf{SE}(3),\ \forall \mathbf{P}_j \in \mathcal{T}_i\end{aligned} \tag{8}$$

where $\mathcal{B}_i$ is the set of the IMU factors of UAV i, $\mathbf{P}_j$ is the j-th pose in $\mathcal{T}_i$, $\mathbf{E}_i$ project the global state $\mathcal{X}$ to local state $\mathcal{X}_i$, $f_{cvio_i}(\mathcal{X}_i)$ is the subproblem of the global optimization Problem (2) on UAV i.

Following Eq. (4)-(6), the iteration update for the collaborative VIO problem (8) is,

$$\begin{aligned}\mathcal{X}_i^{t+1} =& \arg\min_{\mathcal{X}_i} f_{cvio_i}(\mathcal{X}_i) + h(\mathcal{X}_i) \\ & s.t.\ \mathbf{P}_i \in \mathbf{SE}(3),\ \forall \mathbf{P}_i \in \mathcal{T}_i \qquad (9)\\ h(\mathcal{X}_i) =& \frac{1}{2}\sum_{\mathbf{P}_j \in \mathcal{T}_i} \|\log((\mathbf{P}_j^t)^{-1}\mathbf{P}_j^k) + (\tilde{\mathbf{P}}_j^k)^t\|_{\Sigma_{\rho_T}} \\ & + \rho_{\mathcal{L}} \sum_{l_i \in \mathcal{L}_i} \|l_i - l_i^t\| \qquad (10)\end{aligned}$$

$$\mathbf{P}_j^{t+1} = \frac{1}{n_i}\sum_{k=1}^{n_i} (\mathbf{P}_j^k)^{t+1},\ \forall \mathbf{P}_j \in \mathcal{T}_i \qquad (11)$$

$$(\tilde{\mathbf{P}}_j^k)^{t+1} = (\tilde{\mathbf{P}}_j^k)^t + \log((\mathbf{P}_j^{t+1})^{-1}(\mathbf{P}_j^k)^t),\ \forall \mathbf{P}_j \in \mathcal{T}_i, \qquad (12)$$

where $\log(\cdot)$ is the logarithm map of the Lie group SE(3) [57], $(\tilde{\mathbf{P}}_j^k)^t$ is the auxiliary variable, which will be detailed in Sect. VI-B4. $h(\mathcal{X}_i)$ is the consensus item to ensure the local state $\mathcal{X}_i$ is consistent with the global state $\mathcal{X}$. Inside $h(\mathcal{X}_i)$, the first item $\sum_{P\in\mathcal{T}_i} \|\log((\mathbf{P}_i^t)^{-1}\mathbf{P}_i^k) + (\tilde{\mathbf{P}}_i^k)^t\|_{\Sigma_{\rho_T}}$ is to ensure the poses are consistent among UAVs, and the second item $\rho_{\mathcal{L}}\sum_{l_i\in\mathcal{L}_\rangle}\|l_i - l_i^t\|$ is to ensure the convergence of the distributed optimization as proved in [14]. The full algorithm of the distributed optimization for collaborative VIO is shown in Alg. 1.

Subsequent experiments, as detailed in Sect. IX-B, confirm ADMM's suitability for D^2VINS. Importantly, each UAV independently computes Eq. (11). Due to differing neighbor relationships, UAV i focuses only on averaging poses in $\mathcal{X}_i$. While this involves some computational redundancy, its impact on the overall distributed computing performance is minimal. Additionally, data sharing in Alg. 1 is limited to UAVs within the near-field state estimation scope.

Algorithm 1: Distributed Optimization for Collaborative VIO on UAV i

Input: n UAVs, m landmarks, $iter$ iterations, ρ and ρ_T.

Output: n UAVs' local states $\mathcal{X}_i$.

Initialize $\mathcal{X}_i$ and $\mathcal{X}$, $\mathcal{T}_i$, $\mathcal{L}_i$.

for $t = 1$ *to* $iter$ **do**

- Solve the local optimization problem (9) with $\mathcal{X}_i$
- Broadcast the poses $\mathcal{T}_i$ in local state $\mathcal{X}_i$ to all UAVs
- Update $\mathcal{X}_i$ with by averaging poses among swarm with Eq. (11).
- Update the states for auxiliary variables $\{\tilde{\mathbf{P}}_j^k ...\}$ using Eq. (12).

4) Optimization on manifold: One key problem in SLAM is the optimization on the manifold. In Alg. 1, Problem (9) is solved by Riemannian trust region (RTR) method [27] to ensure the pose state is on $\mathbf{SE}(3)$ group, i.e., $\mathbf{P}_i \in \mathbf{SE}(3),\ \forall \mathbf{P}_i \in \mathcal{T}_i$. In addition, the auxiliary states $\{\tilde{\mathbf{P}}_j^k ...\}$ are defined on $\mathfrak{se}(3)$. We use the logarithm map [57] to transform the error $(\mathbf{P}_j^{t+1})^{-1}(\mathbf{P}_j^k)^t$ defined on $\mathbf{SE}(3)$ into $\mathfrak{se}(3)$. The quaternion averaging in Eq. (11) is solved by quaternion averaging algorithm [58], which is well known in the aerospace field.

Our modified ADMM algorithm in Alg. 1 allows states to be defined on SE(3), enabling us to leverage a wide range of foundational tools from VINS research, such as the Riemannian trust-region method (adopted in [12]) and existing marginalization techniques (Sect. VI-D). This approach also helps avoid deviations from the linearization point due to incorrect initialization. In contrast, for pose estimation problem, ARock shall use variables on $\mathfrak{se}3$ (will be detailed in Sect. VII). When using ARock on Problem 8, if the initialization of VIO is suboptimal—which is unavoidable in practice—the results on $\mathfrak{se}3$ may deviate significantly from the linearization point, potentially compromising the accuracy. We believe above factors positively influence the convergence of our algorithm compared to ARock approach, as verified by our early-stage tests. What's more, ADMM's robustness in complex, large-scale bundle adjustment scenarios is well-established [14], [15], leading us to select this approach instead of using ARock.

C. Landmark Management

It is essential to partition landmarks to disjoint sets for each UAV (detailed in II-A2), cap the number of landmarks to balance real-time performance demands, while ensuring sufficient measurements for observability in challenging conditions. Moreover, filtering out outlier measurements increases the system's robustness and reduces the likelihood of false matches. These requirements of landmark partitioning can be formulated as follows, for agent k,

$$\begin{aligned}\max_{\mathcal{L}_k\subseteq\mathcal{L}} \quad & (f_s(\mathcal{L}_k), f_c(\mathcal{L}_k)), && (13)\\ f_s(\mathcal{L}_k) = \quad & w_l m_l(\mathcal{L}_k) + w_r m_r(\mathcal{L}_k) - w_o o(\mathcal{L}_k), && (14)\\ f_c(\mathcal{L}_k) = \quad & \min_{j,t} m(\mathcal{L}_k, \mathcal{K}_j^t) && (15)\\ \text{s. t.} \quad & |\mathcal{L}_k| \le \tau_l && (16)\\ & m_l(\mathcal{L}_k) + m_r(\mathcal{L}_k) \le \tau_m && (17)\\ & \mathcal{L}_k \subseteq \mathcal{LD}_k, && (18)\end{aligned}$$

where $f_s(\mathcal{L}_k)$ is the weighted number of measurements of $\mathcal{L}_k$, $m_l(\mathcal{L}_k)$ and $m_r(\mathcal{L}_k)$ represent the number of measurements of $\mathcal{L}_k$ from local keyframes (keyframes generated by UAV k itself) and remote keyframes, respectively. $m(\mathcal{L}_k, \mathcal{K}_j^t)$ is the number of measurement of $\mathcal{L}_k$ and keyframe $\mathcal{K}_j^t$, and $f_c(\mathcal{L}_k)$ is its lower bound. $\max_{\mathcal{L}_k} f_c(\mathcal{L}_k)$ aims to maximize this lower bound. $o(\mathcal{L}_k)$ is the number of outlier measurements of $\mathcal{L}_k$. The weights w_l, w_r, and w_o dictate the significance of respective measurements. The number of landmarks in $\mathcal{L}_k$ is capped by τ_l, while the maximum number of measurements is restricted by τ_m. $\mathcal{K}_j^t$ refers to a frame within the sliding window. $\mathcal{LD}_k$ represents the landmarks first discovered by UAV k, so the sets $\{\mathcal{LD}_1, \mathcal{LD}_2, \ldots, \mathcal{LD}_n\}$ partition landmarks $\mathcal{L}$ into n disjoint groups and the condition $\mathcal{L}_k \subseteq \mathcal{LD}_k$ ensures that $\mathcal{L}$ is divided into disjoint sets in Problem (13). D^2VINS adopts an 'observed-first' principle for this partitioning (in Eq. (18)), which primarily benefits from single-round communication during complete keyframe broadcasts. While alternative strategies like graph partitioning techniques [59] could potentially enhance optimization, they may introduce

Algorithm 2: Multi-Robot Landmark Selection

Input: Current UAV k, available landmarks $\mathcal{L}$, set of available keyframes $\{\mathcal{F}\}$, a dictionary $\mathcal{O}$ representing the observed landmarks of each frame, with the relationship $\mathcal{O}[\mathcal{F}] \rightarrow \mathcal{L}_f$.

Output: Set of selected landmarks $\mathcal{L}_k$

1 $\mathcal{N}_l \leftarrow \emptyset$
2 $\mathcal{L}_k \leftarrow \emptyset$
3 $\mathcal{O}_a \leftarrow \mathcal{O}$
4 **foreach** $\mathcal{F} \in \{\mathcal{F}\}$ **do**
5 $\mathcal{N}_l[\mathcal{F}] \leftarrow 0$
6 **while** *True* **do**
7 **if** $empty(\mathcal{N}_l)$ **then**
8 break
9 $\mathcal{F} \leftarrow \arg\min_{\mathcal{F} \in \{\mathcal{F}\}} \mathcal{N}_l[\mathcal{F}]$
10 **if** $empty(\mathcal{O}_a[\mathcal{F}])$ **then**
11 $\mathcal{N}_l \leftarrow \mathcal{N}_l \setminus \{\mathcal{F}\}$
12 continue
13 $l \leftarrow \arg\max_{l \in \mathcal{O}_a[\mathcal{F}]} cost(k)$
14 **foreach** $track \in l.tracks$ **do**
15 $\mathcal{N}_l[track.\mathcal{F}] \leftarrow \mathcal{N}_l[track.\mathcal{F}] + 1$
16 $\mathcal{L}_k \leftarrow \mathcal{L}_k \cup \{l\}$
17 **foreach** $\mathcal{F} \in \{\mathcal{F}\}$ **do**
18 $\mathcal{O}_a[\mathcal{F}] \leftarrow \mathcal{O}_a[\mathcal{F}] \setminus \{l\}$
19 **if** $|\mathcal{L}_k| > \tau_l$ *or* $m_l(\mathcal{L}_k) + m_r(\mathcal{L}_k) > \tau_m$ **then**
20 break

extra communication overhead. Therefore, such methods are not implemented in D^2VINS.

Problem (13) is an extended coverage problem, which involves multi-objective optimization. To solve this, we introduce a novel multi-robot landmark selection (MLS) algorithm, detailed in Alg. 2. Within this algorithm, we use the following cost function to approximate Eq. (14):

$$cost(l) = \begin{cases} w_l m_l(l) + w_r m_r(l) - w_o o(l) & \text{if } l \in \mathcal{LD}_k \\ -1 & \text{otherwise,} \end{cases} \tag{19}$$

where $m_l(l)$ and $m_r(l)$ represent the number of measurements of landmark l from local and remote keyframes, respectively, and $o(l)$ is the count of outlier measurements for l.

To select landmarks, our algorithm initially sets up dictionary $\mathcal{N}_l$ to track each landmark's current measurement count and dictionary $\mathcal{O}_a$ to record unselected landmarks observed by each frame. Each iteration begins by identifying the keyframe with the fewest measurements in the sliding window (line 9), aiming to approximate Eq. (15). It then selects a landmark l observed by this keyframe that maximizes Eq. (19). Subsequently, the algorithm updates $\mathcal{N}_l$, incrementing l's measurement count for each corresponding keyframe in $l.tracks$. The selected landmark l is added to $\mathcal{L}_k$, and $\mathcal{O}_a$ is updated to exclude l from each keyframe's observed landmarks. The process repeats until the number of selected landmarks surpasses τ_l, the count of selected measurements reaches τ_m, or no more measurements are available. The resulting set of selected landmarks is $\mathcal{L}_k$.

Given the occasional scarcity of common landmarks among robots, landmarks may be reused across different robots in D^2VINS, a situation that arises when Eq. (18) is not fully met. This reuse increases the computational burden and can slightly compromise accuracy, yet it is vital for the system's robustness. Our experiments have shown that the system can become unstable or even diverge when measurements per frame are few, necessitating a compromise in Alg. 2 for stability. In practice, we set $w_l = 1$ and $w_r = 2$ to encourage the addition of more cross-robot feature points and $w_o = 2$ to reduce outliers, identified by large reprojection errors prior to optimization.

D. Sliding Window & Marginalization

1) Sliding window: In D^2VINS, each UAV independently manages its keyframes within a sliding window, akin to the approach in [12]. Upon adding a new frame, if it is not a keyframe, the second newest frame is discarded; otherwise, the oldest keyframe is removed. Following each update of the sliding window, the latest information is shared with other UAVs in the swarm.

2) Management of remote keyframes: Successful multi-robot sparse feature matching, as detailed in Sect. V-D, results in the inclusion of the remote frame's pose ${}^{v_k}\hat{\mathbf{T}}_j^t$ into the local state vector $\mathcal{X}_i$ for optimization in Problem (8). Upon receiving updated sliding window data from other UAVs, D^2SLAM removes any remote keyframes from UAV k that are no longer within its local sliding window. This removal, akin to local keyframe deletion, leads to marginalization, which will be discussed further.

3) Marginalization: When discarding old keyframes, each subproblem $f_{cvio_i}(\mathcal{X}i)$ of Problem (8) is linearized, and a new set of priors $\{(\mathbf{r}_{p_0}, \mathbf{H}_{p_0}), (\mathbf{r}_{p_1}, \mathbf{H}_{p_1}) \cdots (\mathbf{r}_{p_{N-1}}, \mathbf{H}_{p_{N-1}})\}$ is computed using the Schur complement to exclude old states. This process, known as marginalization [12], [60], treats each subproblem independently and is therefore executed distributively. Since Problem (8) involves states averaged across UAVs (Eq. 11), the marginalization process maintains consistency across UAVs, meaning states are linearized at similar or identical points on different UAVs.

E. Initialization

D^2VINS initialization involves two phases: local keyframe initialization and remote UAV initialization. For local keyframes, utilizing stereo or multi-camera setups, we employ triangulation for landmark position initialization where measurements have a sufficient baseline (typically 0.05cm) from multiple cameras or due to motion. Additionally, IMU predictions initialize poses of subsequent local UAV Keyframes. To enhance stability, priors are added to the unobservable components (x, y, z, yaw) of the first keyframe. In multi-UAV scenarios, PnP RANSAC (or UPnP) is used to initialize the pose of remote keyframes when their coordinate system references differ from the local UAV.

F. Implementation

D^2SLAM solves Problem (9) using Ceres-solver [61], applying a Dogleg strategy [62] and a dense Schur solver. For enforcing pose manifold constraints, we utilize the Local Parameterization feature in Ceres, essentially implementing the RTR (Riemannian Trust-Region) method [27].

Alg. 1, due to Eq. (11), operates synchronously, requiring UAVs to wait for others' results each iteration, which can be less efficient in poor communication environments. However, we've found that relaxing the synchronization condition of Eq. (11) — using the most recently received state averages without waiting — also produces results close to the synchronous approach. We label this method as asynchronous D^2VINS, and a comparative analysis with its synchronous counterpart will be presented in Sect. IX-B.

VII. Far-field state estimation: D^2PGO

Far-field estimation in D^2SLAM encompasses both global trajectory consistency (D^2PGO) and real-time global odometry. We model the global trajectory estimation using a pose graph approach [63], where keyframes are graph nodes and edges represent relative poses between them. This pose graph is formulated as a factor graph, treating measurements as Gaussian-distributed, thus constituting the classic pose graph optimization problem [19]. D^2PGO employs a two-stage optimization process to decentralize and distribute the pose graph optimization across multiple UAVs. The pose graph optimization problem is,

$$\begin{aligned}\min_{\mathbf{x}_i} \sum_{(i_{t_0},j_{t_1})\in\mathcal{E}} &\left\| {}^{v_k}\hat{\mathbf{p}}_j^{t_1} - {}^{v_k}\hat{\mathbf{p}}_i^{t_0} - {}^{v_k}\hat{\mathbf{R}}_i^{t_0}\mathbf{z}_{\mathbf{p}i\to j}^{t_0\to t_1}\right\|_{\Sigma_t}^2 \\ &+ \left\| {}^{v_k}\hat{\mathbf{R}}_j^{t_1} - {}^{v_k}\hat{\mathbf{R}}_i^{t_0}\mathbf{z}_{\mathbf{R}i\to j}^{t_0\to t_1}\right\|_{\Sigma_R}^2, \\ s.t.\ &{}^{v_k}\hat{\mathbf{R}}_i^t \in SO(3),\end{aligned} \tag{20}$$

where $\mathcal{E}$ is the set of all edge, including loop closure edge (generated by loop closure detection) and ego-motion edge (generated by D^2VINS), $\mathbf{z}_{\mathbf{p}i\to j}^{t_0\to t_1}, \mathbf{z}_{\mathbf{R}i\to j}^{t_0\to t_1}$ are the rotation part and translation part of the relative pose of the edge (i_{t_0}, j_{t_1}), respectively. ${}^{v_k}\mathbf{R}_i^t$ is the rotation part of ${}^{v_k}\hat{\mathbf{T}}_i^t$, and ${}^{v_k}\mathbf{p}_i^t$ is the translation part of ${}^{v_k}\hat{\mathbf{T}}_i^t$, $\mathbf{x}$ is the full state of the pose graph:

$$\begin{aligned}\mathcal{X} = \Big[&{}^{v_k}\hat{\mathbf{T}}_0^{t_0}, {}^{v_k}\hat{\mathbf{T}}_0^{t_1}, \dots {}^{v_k}\hat{\mathbf{T}}_0^{t_{M-1}}, \dots\ {}^{v_k}\hat{\mathbf{T}}_{N-1}^{t_0}, \\ &{}^{v_k}\hat{\mathbf{T}}_{N-1}^{t_1}, \dots {}^{v_k}\hat{\mathbf{T}}_{N-1}^{t_{M-1}}\Big],\end{aligned} \tag{21}$$

where N is the number of UAVs, M is the number of keyframes of each UAV.

By reformulating the pose graph optimization problem (20) into a distributed framework (3), the equation becomes:

$$\begin{aligned}&\min_{\mathbf{x}_i} \sum_{i=0}^{N-1} f_{pgo_i}(\mathbf{x}_i) \\ f_{pgo_i}(\mathbf{x}_i) = &\sum_{(i_{t_0},j_{t_1})\in\mathcal{E}^i} \left\| {}^{v_k}\hat{\mathbf{p}}_j^{t_1} - {}^{v_k}\hat{\mathbf{p}}_i^{t_0} - {}^{v_k}\hat{\mathbf{R}}_i^{t_0}\mathbf{z}_{\mathbf{p}i\to j}^{t_0\to t_1}\right\|_{\Sigma_p}^2 \\ &+ \left\| {}^{v_k}\hat{\mathbf{R}}_j^{t_1} - {}^{v_k}\hat{\mathbf{R}}_i^{t_0}\mathbf{z}_{\mathbf{R}i\to j}^{t_0\to t_1}\right\|_{\Sigma_R}^2, \\ &s.t.\ {}^{v_k}\hat{\mathbf{R}}_i^t = \mathbf{E}_{i_t}^{\mathbf{R}}\mathbf{z}, {}^{v_k}\hat{\mathbf{p}}_i^t = \mathbf{E}_{i_t}^p\mathbf{z}, {}^{v_k}\mathbf{R}_i^t \in SO(3),\end{aligned} \tag{22}$$

where x_i is the state of the UAV i, $\mathbf{E}_{i_t}^{\mathbf{R}}\mathbf{z}, \mathbf{E}_{i_t}^p$ project global state z to local states, $\mathcal{E}^i$ is the set of edges of the UAV i.

D^2PGO's initial stage employs rotation initialization for the pose graph to circumvent local minima. The second stage focuses on refining the pose graph optimization. Beyond optimization, an outlier rejection module is implemented to filter anomalies in the pose graph problem (20). The final step involves merging the pose graph optimization results with the VIO results to establish each UAV's globally consistent real-time odometry:

$${}^{v_k}\hat{\mathbf{T}}_i^{t_1} = {}^{v_k}\hat{\mathbf{T}}_i^t(\tilde{\mathbf{T}}_i^t)^{-1}\tilde{\mathbf{T}}_i^{t_1}, \tag{23}$$

where ${}^{v_k}\hat{\mathbf{T}}_i^t$ denotes the latest keyframe pose of UAV i in D^2PGO, $\tilde{\mathbf{T}}_i^t$ represents the odometry estimation from D^2VINS for this keyframe, and $\tilde{\mathbf{T}}_i^{t_1}$ is the real-time odometry estimation with D^2VINS.

A. ARock for Distributed Optimization

To solve the PGO, we employ ARock [16], an asynchronous distributed optimization algorithm. ARock iteratively updates the distributed optimization problem (3) as described in [16]:

$$\hat{\mathbf{x}}_i^t = \arg\min_{\mathbf{x}_i} f_i(\mathbf{x_i}) + \mathbf{x_i}\sum_{r\in\mathcal{R}(i)} z_{ri,r}^t + \frac{\gamma}{2}|\mathcal{R}(i)|\cdot\|\mathbf{x_i}\|^2 \tag{24}$$

$$z_{ri,i}^{t+1} = z_{ri,i}^t - \eta_k((z_{ri,i}^t + z_{ri,r})/2 + \gamma\hat{\mathbf{x}}_i^t)\ \forall r\in\mathcal{R}(i), \tag{25}$$

where $\mathcal{R}(i)$ represents the set of neighbors for agent i, $(\cdot)^t$ denotes the state in the t-th iteration, and η_k and γ are two parameters. Dual variables $z_{ij,i}$ and $z_{ij,j}$ for agents i and j are updated independently by each, and $z_{ij,i} = z_{ji,i}, z_{ij,j} = z_{ji,j}$. We modify the update rule by substituting dual variables with $y_{ij,i} = -z_{ij,i}/\gamma$ and $y_{ij,j} = -z_{ij,j}/\gamma$ to obtain:

$$\hat{\mathbf{x}}_i^t = \arg\min_{\mathbf{x}_i} f_i(\mathbf{x_i}) + \sum_{r\in\mathcal{R}(i)} \frac{\gamma}{2}\|\mathbf{x_i} - y_{ri,r}^t\|^2 \tag{26}$$

$$y_{ri,i}^{t+1} = y_{ri,i}^t - \eta_k((y_{ri,i}^t + y_{ri,r})/2 - \hat{\mathbf{x}}_i^t)\ \forall r\in\mathcal{R}(i) \tag{27}$$

The complete ARock algorithm for decentralized optimization is outlined in Alg. 3. Unlike the synchronous Alg. 1, ARock operates asynchronously, utilizing the latest received dual variables from remote agents without the need for synchronous execution as per Eq. (26)-(27), making it less sensitive to communication delays. Additionally, ARock is proven to have linear convergence [16].

Algorithm 3: ARock for Distributed Optimization

Input: n UAVs, m landmarks, $iter$ iterations.
Initialize $y^0_{ij,i}, y^0_{ij,j}$ for all i, j to 0.
for $t = 1$ *to* $iter$ **do**
  Solve the local optimization problem (26)
  Update the dual states $y^t_{ri,i}$ with Eq. (27).
  Broadcast the dual state $y^t_{ri,i}$ to all UAVs
  if *Convergence* **then**
    Break

B. Asynchronous Distributed Rotation Initialization

Nonlinearities in the pose graph problem primarily arise from rotations. With proper rotation initialization, the problem closely resembles a linear least squares issue, simplifying its solution. This technique of rotation initialization is known to effectively avoid local minima in pose graph optimization [64]. The goal of this initialization is to find an exact or approximate solution for the rotational component of Problem (20), a task akin to solving the rotation averaging problem [65]:

$$\begin{aligned}\min_{\mathbf{x}_i} \sum_{(i_{t_0}, j_{t_1})\in\mathcal{E}^i} \left\| {}^{v_k}\hat{\mathbf{R}}^{t_1}_j - {}^{v_k}\hat{\mathbf{R}}^{t_0}_i \mathbf{z}_{\mathbf{R}}{}^{t_0\to t_1}_{i\to j} \right\|^2_{\Sigma_R} \\ s.t.\ {}^{v_k}\hat{\mathbf{R}}^t_i \in SO(3).\end{aligned} \tag{28}$$

Chordal relaxation [66] is an effective algorithm for solving the rotation initialization problem. It starts by relaxing the $\mathbf{SO}(3)$ constraint in (28):

$$\begin{aligned}\min_{\mathbf{x}_i} \sum_{(i_{t_0}, j_{t_1})\in\mathcal{E}} \left\| {}^{v_k}\overline{\mathbf{R}}^{t_1}_j - {}^{v_k}\overline{\mathbf{R}}^{t_0}_i \mathbf{z}_{\mathbf{R}}{}^{t_0\to t_1}_{i\to j} \right\|^2_{\Sigma_R} \\ + \sum_{\overline{R}^t_i\in\mathcal{R}_i} \|{}^{v_k}(\overline{\mathbf{R}}^t_i)_3 - \mathbf{v}_i\|_{\Sigma_g},\end{aligned} \tag{29}$$

where $\mathcal{R}_i$ denotes all rotations in $\mathbf{x}_i$, and $\overline{\mathbf{R}}^t_i$ is a 3x3 matrix representing the rotation of UAV i at time t. The term $\|{}^{v_k}(\overline{\mathbf{R}}^t_i)_3 - \mathbf{v}_i\|_{\Sigma_g}$ is the vertical prior, added because the roll-pitch angle from VIO is observable and can be used as a prior, enhancing initialization [64]. Here, ${}^{v_k}(\overline{\mathbf{R}}^t_i)_3$ is the third row of $\overline{\mathbf{R}}^t_i$, $\mathbf{v}_i = (\tilde{\mathbf{R}}^t_i)^T\mathbf{g}$ with $g = [0, 0, 1]^T$, and $\tilde{\mathbf{R}}^t_i$ is the odometry output rotation of UAV i at time t. Problem (29) is a linear least squares problem that can be efficiently solved with a linear solver. Then, we can recover the rotation matrix by

$$\begin{aligned}{}^{v_k}\hat{\mathbf{R}}^t_i = & argmin_{\hat{\mathbf{R}}} \|\hat{\mathbf{R}} - {}^{v_k}\overline{\mathbf{R}}^t_i\|^2_F, \\ & s.t.\ \hat{R}\in\mathbf{SO}(3)\end{aligned} \tag{30}$$

where $\|\cdot\|_F$ is the Frobenius norm. Problem (30) has a closed form soluting with SVD decomposition [65]:

$${}^{v_k}\breve{\mathbf{R}}^t_i = \mathbf{S}diag([1, 1, det(\mathbf{SV^T})])\mathbf{V^T}, \tag{31}$$

where ${}^{v_k}\breve{\mathbf{R}}^t_i \in SO(3)$ is the initial result of ${}^{v_k}\hat{\mathbf{R}}^t_i$, $\mathbf{SDV^T}$ is the SVD decomposition of $\overline{\mathbf{R}}^t_i$.

Algorithm 4: Asynchronous Distributed Rotation Initialization for UAV i

Input: ϵ for termination, max_iter iterations, min_iter to avoid early exiting.
Initialize $y^0_{ij,i}, y^0_{ij,j}$ for all i, j to 0.
for $t = 1$ *to* max_iter **do**
  Solve the local optimization problem (26) with f_i defined from (32)
  Update the dual states $y^t_{ir,i}$ with Eq. (27).
  Broadcast the dual state $y^t_{ir,i}$ to neighbors
  if $\frac{1}{|\mathcal{R}_i|}\sum_{\overline{\mathbf{R}}\in\mathcal{R}_i} \|(\overline{\mathbf{R}})^t - (\overline{\mathbf{R}})^{t-1}\|_F / \|(\overline{\mathbf{R}})^t\|_F < \epsilon$ ***and*** $iter > min_iter$ **then**
    break
for $\overline{R}\in\mathcal{R}_i$ **do**
  Recover the rotation ${}^{v_k}\hat{\mathbf{R}}^t_i$ with Eq. (31)

Algorithm 5: Asynchronous Distributed Pose Graph Optimization for UAV i

while *Not shutdown* **do**
  if *New data received* **then**
    Adding new states or cost functions to optimizer.
  Solve the local optimization problem (26) with f_i defined from (33)
  Update the dual states $y^t_{ir,i}$ with Eq. (27).
  Broadcast the dual state $y^t_{ir,i}$ to neighbors
  Recover the poses with Eq. (34)
  Sleep for a while

In D^2PGO, we solve Problem (29) distributively, redefining it as:

$$\begin{aligned}\min_{\mathbf{x}_i} \sum_{i=0}^{N-1} \Bigg\{ \sum_{(i_{t_0}, j_{t_1})\in\mathcal{E}_i} \left\| {}^{v_k}\overline{\mathbf{R}}^{t_1}_j - {}^{v_k}\overline{\mathbf{R}}^{t_0}_i \mathbf{z}_{\mathbf{R}}{}^{t_0\to t_1}_{i\to j} \right\|^2_{\Sigma_R} \\ + \sum_{\overline{R}^t_i\in\mathbf{x}_i} \|{}^{v_k}(\overline{\mathbf{R}}^t_i)_3 - \mathbf{v}_i\|_{\Sigma_g} \Bigg\},\end{aligned} \tag{32}$$

Utilizing Alg. 3, we address problem (32) through our asynchronous distributed rotation initialization algorithm, detailed in Alg. 4. The local optimization problem within Alg. 4 (converting Eq. (32) into Eq. (26)) is a linear least squares problem, efficiently solvable via Cholesky factorization [67]. Iteration termination occurs when changes in the optimized state become negligible, as indicated in Alg. 4 line 6, with $(\overline{\mathbf{R}})^t$ representing the post-t-iteration state. Additionally, to avoid premature termination (e.g., due to initial data absence), a minimum number of iterations is also set.

C. Asynchronous Distributed Pose Graph Optimization

After completing the initialization, we build the perturbation problem of (22) with the initialization rotation,

$$\begin{aligned}
&\min_{\mathbf{x}_i^*} \sum_{i=0}^{N-1} f_{pgo_i}^*(\mathbf{x}_i^*) \\
f_{pgo_i}^*(\mathbf{x}_i^*) = &\sum_{(i_{t_0},j_{t_1})\in\mathcal{E}^i} \left\| {}^{v_k}\hat{\mathbf{p}}_j^{t_1} - {}^{v_k}\hat{\mathbf{p}}_i^{t_0} - \right. \\
&\boxplus\left({}^{v_k}\breve{\mathbf{R}}_i^{t_0}, {}^{v_k}\delta\theta_i^t\right)\mathbf{z}_{\mathbf{p}}{}_{i\to j}^{t_0\to t_1}\Big\|_{\Sigma_p}^2 + \\
&\left\|\boxplus({}^{v_k}\breve{\mathbf{R}}_j^{t_1}, {}^{v_k}\delta\theta_j^{t_1}) - \boxplus({}^{v_k}\breve{\mathbf{R}}_i^{t_0}, {}^{v_k}\delta\theta_i^t)\mathbf{z}_{\mathbf{R}}{}_{i\to j}^{t_0\to t_1}\right\|_{\Sigma_R}^2, \\
&s.t.\ {}^{v_k}\delta\theta_i^t = \mathbf{E}_{i_t}^{\mathbf{R}}\mathbf{z}^*, {}^{v_k}\hat{\mathbf{p}}_i^t = \mathbf{E}_{i_t}^{p}\mathbf{z}^*,
\end{aligned} \tag{33}$$

where $({}^{v_k}\hat{\mathbf{p}}_i^{t_0}, {}^{v_k}\delta\theta_i^{t_0})$ is the perturbation state of ${}^{v_k}\mathbf{T}_i^{t_0}$, where ${}^{v_k}\hat{\mathbf{p}}_i^{t_0}$ is the position of ${}^{v_k}\mathbf{T}_i^{t_0}$ and ${}^{v_k}\delta\theta_i^{t_0} \in \mathfrak{so}3$ is the perturbation state of the rotation. x_i^* is the the perturbation state of $\mathbf{x}^i$, and $\mathbf{z}^*$ is the global perturbation state, $\boxplus(\mathbf{R},\delta\theta) = \mathbf{R}Exp(\delta\theta_i^{t_0})$ retracts $\mathfrak{so}3$ vector to $\mathbf{SO}(3)$ at $\mathbf{R}$ [27], where $Exp(\cdot)$ maps $\mathfrak{so}3$ to $SO(3)$ [68]. Contrasting with the two-step PGO approach in Eq (10) of [4], our Problem (33) is nonlinear and circumvents the accuracy issues associated with the linearization used in [4].

Our algorithm employs Alg. 3 to address (33), ensuring asynchronicity. Transforming $f_{pgo_i}^*(\mathbf{x}_i^*)$ per (26) yields a nonlinear least squares problem, solvable via the Levenberg-Marquardt algorithm [69]. Convergence is determined by observing changes in the cost function of $f_{pgo_i}^*(\mathbf{x}_i^*)$. As in Alg. 4, a minimum iteration count is set to avoid premature termination. After convergence, we recover the global pose with

$${}^{v_k}\hat{\mathbf{T}}_i^{t\,v_k} = \begin{bmatrix} \boxplus({}^{v_k}\breve{\mathbf{R}}_i^{t_0}, {}^{v_k}\delta\theta_i^t) & \hat{\mathbf{p}}_i^t \\ 0 & 1 \end{bmatrix} \tag{34}$$

Our asynchronous distributed pose graph optimization (ARockPGO) is outlined in Alg. 5. Unlike Alg. 4, ARockPGO operates continuously throughout UAV flight, updating at a fixed frequency (typically 1Hz) and incrementally integrating new information. This approach balances convergence speed against communication load.

D. Convergence

ARock demonstrates strong convergence performance, suggesting it can potentially reach optimality in the initial stage of linear least squares problems. The effectiveness of rotation initialization in preventing convergence to incorrect local optima [64] enhances D^2PGO's ability to achieve or closely approximate the global optimum.

E. Outlier Rejection

Currently, the mainstream outlier rejection methods for multi-robot PGO include Pairwise Consistent Measurement Set Maximization (PCM) [1], [3], [70] and Graduated Non-Convexity (GNC) [71]. PCM, independent of back-end optimization, is efficient but less effective with large pose graphs. Conversely, GNC is integrated into back-end optimization, offering higher efficiency but operating synchronously. In D^2PGO, we adopt a distributed version of PCM as used in [1], [3], mitigating PCM's efficiency issues at scale. We believe this approach provides adequate performance for PGO that is not real-time critical.

F. Implementation

The initial stage of D^2PGO is designed to ensure accurate trajectory estimation even in highly noisy environments. However, in real-world applications, this stage is often unnecessary, as VIO's yaw measurements are typically precise, allowing for its omission in practice.

D^2PGO is implemented in C++, utilizing Eigen [72]'s LLT algorithm for the linear least squares calculation in rotation initialization. For the second stage, we employ the Levenberg-Marquardt algorithm from Ceres-solver [61].

VIII. System Setup

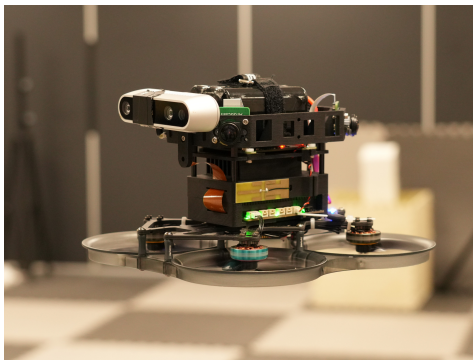


Fig. 10: Our aerial platform equipped with quad fisheye cameras, an onboard computer, an all-in-one flight controller, an additional Intel RealSense d435i camera, and a 6s 18650 battery.

Our experimental aerial swarm is based on custom-modified aerial robots, as depicted in Fig. 10. Each UAV is a modified 3.5-inch cinewhoop FPV UAV, equipped with a quartet of fisheye cameras providing 360-degree horizontal visibility. They are powered by NVIDIA Xavier NX onboard computers, boasting robust GPU capabilities. To minimize takeoff weight, we use a lightweight, integrated flight controller with four electronic speed controls (ESCs) and customized PX4 [74] firmware. The UAV's takeoff weight is approximately 623g with a 22.8v 1550mah battery, offering a maximum endurance of about 10 minutes, which suffices for our tests. Optionally, an Intel RealSense d435i camera can be mounted on the UAV.

D^2SLAM is implemented in C++ and utilizes ONNX Runtime [75] for CNN inference. We activate TensorRT for enhanced performance, employing the int8 mode on the onboard computer to optimize speed. Additionally, we develop TaichiSLAM, a GPU-accelerated mapping module, serving as the dense mapping backend for later experiments. TaichiSLAM, crafted in the high-performance Taichi language [76], integrates various algorithms like OctoMap [77], TSDF & ESDF [78], and submap fusion [79] to create globally consistent maps. However, as dense mapping algorithms are not the primary focus of D^2SLAM, they are not elaborated in detail.

TABLE I: The table displays the datasets used for validation. The numbers added as suffix to the datasets correspond to the number of robots involved.

Dataset	Robot Number	Sensors	Movement	Groundtruth	Source
TUM ROOM 2-5	2 to 5	Stereo Camera IMU	HandHold	Yes	[73]
TUM Corr 2-5	2 to 5			Partial	
HKUST RI 3	3	Stereo Camera Depth Camera IMU		No	Customized
Omni 2-5	2 to 5	Quad fisheye cameras IMU	Flight	Yes	
OmniLongNoYaw 5	5		Flight with Yaw Angle Fixed		
OmniLongYaw 5	5		Flight with Yaw Angle Moving		

IX. Experiments

A. Datasets & Evaluation Setup

In the evaluation presented in this section, we test D^2SLAM using various publicly available and customized datasets. These datasets use either stereo cameras or quad cameras as input and partly used a motion capture system to capture ground truth data, as shown in Tab. I. To simulate the scenario of multiple robots operating simultaneously, we align multiple segments of data into a single multi-robot dataset.

Details on the evaluation metrics are available in [1]. Notably, the term *Relative Error (RE)* in this context refers to the accuracy of relative state estimation among UAVs. In the following tables, position measurements are in meters and rotation in degrees.

In real-time scenarios, we do not require distributed optimization to fully converge, similar to some single-robot SLAM systems like VINS-Mono [12], which typically limits its backend optimization to just 8 iterations. Synchronous D^2VINS performs 4 iterations per solution of Eqs. (9) - (12), while asynchronous D^2VINS requires only a single run of these equations per optimization. Given that D^2VINS continuously integrates new data and commences optimization from the previous results, with most of the states from the last optimization still included in $\mathcal{X}_i$, Eqs. (9) - (12) effectively experience multiple iterations over time in the asynchronous D^2VINS framework. D^2PGO's handling in real-time systems is similar to asynchronous D^2VINS.

B. Evaluation of D^2VINS

Our initial tests focused on D^2VINS to assess its near-field state estimation performance. These tests did not rely on ground truth for initializing D^2VINS. We also compared D^2VINS with its single-robot variant, referred to as D^2VINS (single), and with the state-of-the-art VIO approach, VINS-Mono [12]. For alignment, VINS-Mono and D^2VINS (single) are synchronized using the initial pose from ground truth, mirroring the known UAV departure points setup used in [10].

Table II illustrates that our method achieves centimeter-level localization accuracy on the TUM ROOM dataset. Particularly during the first 30 seconds, where the common FoV is favorable, the relative localization accuracy is highly precise at 2.5cm. Even without common FoV for most of the evaluation, our dataset maintains similar centimeter-level accuracy as demonstrated in state-of-the-art works [1], [11]. This evaluation underscores D^2VINS's capability for accurate near-field state estimation using stereo cameras, especially with adequate common field of view, highlighting its potential for stereo camera-equipped UAV platforms.

We also test D^2VINS on custom datasets using omnidirectional cameras, with results detailed in Table III. Our method consistently achieves centimeter-level relative localization accuracy across scenarios ranging from 2 to 5 UAVs. However, we observe a slight decrease in relative localization accuracy as the distance between UAVs increased, as indicated in Table III. Despite this, the accuracy remains sufficient for inter-UAV collision avoidance. Notably, while both D^2VINS and VINS-Mono exhibit drift from ground truth over long flights with yaw rotation (as shown by the large ATE in Table III), particularly in the OmniLongYaw 5 scenario, D^2VINS still maintains centimeter-level relative localization accuracy. This underscores its capability for reliable relative localization in extended flight durations.

In scenarios with sufficient common FoV, D^2VINS not only matches but often surpasses the state-of-the-art relative localization accuracy reported in [1], [11], while also providing superior ego-motion estimation accuracy (evidenced by better ATE compared to VINS-Mono). In cases where common FoV is inconsistent, D^2VINS's relative localization accuracy is comparable to that in [1], [11]. Notably, our results outperform both VINS-Mono and D^2VINS (single) across almost all metrics in Tables II and III, demonstrating our algorithm's superiority in multi-robot scenarios. Specifically, the drift observed in VINS-Mono and D^2VINS (single) can significantly degrade their relative localization accuracy over extended periods, a drawback not evident in our approach.

Furthermore, Table II and Table III present accuracy comparisons of asynchronous D^2VINS (labeled as D^2VINS (async) in the tables). We observe that its performance closely parallels that of synchronous methods. However, in some instances, asynchronous D^2VINS exhibits slight instability in ATE. Given the communication benefits of asynchronous methods, opting for their deployment in real-world environments represents a reasonable compromise.

C. Evaluation of D^2PGO

Table IV showcases D^2PGO's optimization results for various pose graphs generated by the front-ends of D^2SLAM and

TABLE II: The table displays the statistical comparison of D^2VINS against VINS-Mono and D^2VINS (single) on the TUM VI room dataset, where ATE and RE are defined as [1]. These datasets exhibit good common FoV during the initial 30 seconds, hence the results are separately presented for this duration, marked as (30s).

Dataset	Avg. Traj. Len.	Method	ATE_{pos}	ATE_{rot}	RE_{pos}	RE_{rot}
TUM ROOM 3 (30s)	18.3	D^2VINS	**0.030**	0.99	**0.025**	0.77
		D^2VINS (async)	0.035	1.63	**0.025**	0.74
		D^2VINS (single)	0.054	2.44	0.055	1.72
		VINS-Mono	0.055	**0.66**	0.072	**0.52**
TUM ROOM 3	141.6	D^2VINS	0.128	2.40	0.119	2.95
		D^2VINS (async)	**0.111**	2.63	**0.098**	3.08
		D^2VINS (single)	0.181	3.16	0.161	2.91
		VINS-Mono	0.139	**1.49**	0.212	**2.11**
TUM ROOM 4 (30s)	19.1	D^2VINS	0.057	2.20	**0.020**	0.74
		D^2VINS (async)	0.124	1.73	0.038	0.78
		D^2VINS (single)	0.067	3.95	0.064	1.86
		VINS-Mono	**0.056**	**0.67**	0.063	**0.52**
TUM ROOM 4	100.9	D^2VINS	0.115	2.44	**0.080**	1.72
		D^2VINS (async)	**0.105**	2.18	0.082	1.85
		D^2VINS (single)	0.178	4.40	0.144	2.88
		VINS-Mono	0.125	**1.37**	0.156	**1.55**
TUM ROOM 5 (30s)	19.2	D^2VINS	0.079	1.35	0.039	0.73
		D^2VINS (async)	0.108	4.50	**0.037**	0.89
		D^2VINS (single)	0.068	3.81	0.090	1.73
		VINS-Mono	**0.053**	**0.69**	0.058	**0.65**
TUM ROOM 5	103.5	D^2VINS	**0.090**	1.85	**0.073**	**1.57**
		D^2VINS (async)	0.134	4.05	0.091	2.10
		D^2VINS (single)	0.194	4.26	0.183	3.28
		VINS-Mono	0.164	**1.62**	0.209	2.28

TABLE III: The table presents the statistical comparison of D^2VINS with D^2VINS (single) and VINS-Mono [12] using Omni datasets. It also includes the average distance (Avg. Dis.) between UAVs.

Dataset	Avg. Traj. Len.	Avg. Dis.	Method	ATE_{pos}	ATE_{rot}	RE_{pos}	RE_{rot}
Omni 2	49.1	0.68	D^2VINS	**0.069**	**0.85**	**0.075**	0.32
			D^2VINS (async)	0.103	4.41	0.088	**0.21**
			D^2VINS (single)	0.139	4.65	0.161	0.31
			VINS-Mono	0.235	6.08	0.264	0.52
Omni 5	45.4	1.03	D^2VINS	0.075	**0.71**	**0.076**	**0.37**
			D^2VINS (async)	**0.061**	1.10	0.078	0.38
			D^2VINS (single)	0.160	5.56	0.225	0.47
			VINS-Mono	0.282	4.21	0.331	0.61
OmniLongNoYaw 5	213.7	0.48	D^2VINS	0.122	0.83	0.043	0.53
			D^2VINS (async)	**0.118**	**0.71**	**0.031**	0.51
			D^2VINS (single)	0.150	4.36	0.109	**0.39**
			VINS-Mono	0.500	1.78	0.307	1.26
OmniLongYaw 5	237.0	0.38	D^2VINS	2.322	11.09	0.028	**0.65**
			D^2VINS (async)	2.752	18.32	**0.027**	0.72
			D^2VINS (single)	**1.366**	**6.06**	1.019	1.22
			VINS-Mono	4.865	12.48	1.466	3.40

D^2VINS. During the evaluation, we incorporated a consistent 50ms communication delay to emulate real-world conditions. The comparison of D^2PGO's performance, with and without rotation initialization, reveals that rotation initialization plays a crucial role in preventing D^2PGO from converging to local optima.

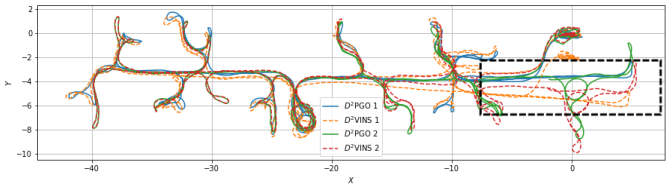


Fig. 11: The figure illustrates D^2SLAM's estimated trajectories on the TUM Corr 2 Dataset, a combination of TUM VI corridor datasets 1, 2. In this dataset, two robots sequentially traverse the same corridor, marked by the black box section. Within this section, the trajectories from D^2VINS exhibit significant drift, whereas those from D^2PGO show minimal drift and maintain good alignment between the two robots.

D. Evaluation of D^2SLAM

We further evaluated D^2SLAM on both public and custom datasets to assess the performance of our near- and far-field state estimation. Leveraging the high accuracy of VIO's angle measurement and aiming for optimal real-time performance, only the second stage of D^2PGO was utilized in these tests. We conducted a comparison between D^2SLAM and the state-of-the-art distributed CSLAM system DOOR-SLAM [3] using

TABLE IV: This table compares D^2PGO with and without rotation initialization, detailing both initial and final costs, as well as the size of the pose graph, which includes the counts of keyframes and edges.

Dataset	Avg. Traj. Len.	Keyframes	Edges	Method	Initial Cost	Final Cost	Iterations	Solve Time (s)
TUM Corr 5	255.6	12065	16814	D^2PGO	350408.6	61.2	234.2	10.1
				D^2PGO (NoRotInit)		1920.4	238.4	10.1
OmniLongYaw 5	242.6	16430	51358	D^2PGO	520424.1	1366.3	37.2	20.5
				D^2PGO (NoRotInit)		41072.3	31.6	20.4
HKUST RI 3	164.5	3347	3677	D^2PGO	6438.5	12.1	152.0	3.0
				D^2PGO (NoRotInit)		1301.8	197.0	3.0

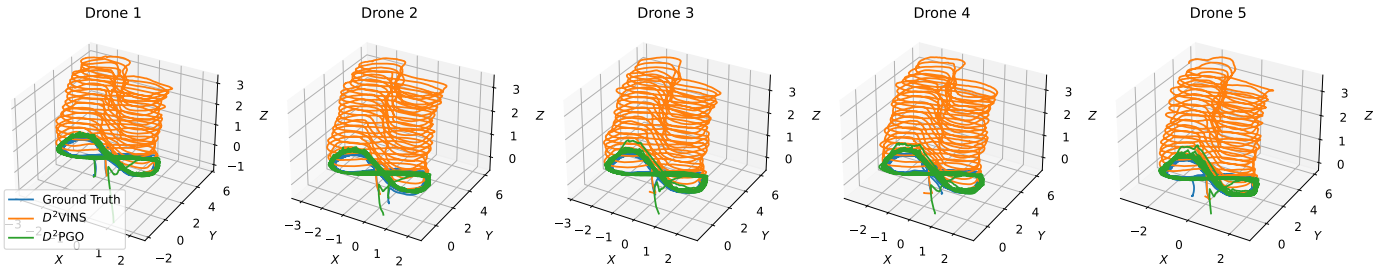


Fig. 12: The images display trajectories estimated by D^2VINS and D^2PGO in the OmniLongYaw 5 dataset. The trajectories from D^2VINS show noticeable drift, whereas those from D^2PGO remain closely aligned with the ground truth.

TABLE V: The table presents statistical results comparing D^2SLAM with DOOR-SLAM [3] on our custom datasets. The estimated trajectories and ground truth are aligned using multiple states [42], based on the first robot's estimations.

Dataset	Method	ATE_{pos}	ATE_{rot}	RE_{pos}	RE_{rot}
OmniLongYaw 5	D^2SLAM	**0.168**	3.33	**0.037**	0.91
	DOOR-SLAM	0.149	**3.47**	0.085	**0.72**
TUM ROOM 5	D^2SLAM	**0.032**	**0.72**	**0.079**	1.90
	DOOR-SLAM	0.064	1.47	0.122	**1.86**
TUM Corr 5	D^2SLAM	0.275	3.93	0.206	**1.77**
	DOOR-SLAM	**0.068**	**1.26**	**0.145**	2.39

datasets with ground truth, with results presented in Table V. For this comparison, we applied our custom implementation in D^2SLAM to generate PGO for DOOR-SLAM, ensuring consistency and eliminating potential discrepancies from different frontends.

Table V shows that our method outperforms DOOR-SLAM in relative state estimation on the first two datasets. This advantage arises because DOOR-SLAM relies on pose graph optimization, a common but loosely coupled approach in CSLAM. While both methods demonstrate comparable global consistency accuracy due to similar inputs, DOOR-SLAM exhibits superior performance on the TUM Corr dataset. This can be attributed to the partial ground truth available only at the beginning and end of the TUM Corr dataset, coupled with inadequate yaw overlap between UAVs towards the dataset's end.

We conduct further tests of D^2SLAM on larger-scale datasets, including TUM VI Corr 5, HKUST RI 3, and OmniLongYaw 5, to evaluate its global consistency. The real-time odometry and final pose graphs estimated in these experiments are illustrated in Fig. 11, Fig. 1b, and Fig. 12. The figures reveal a stark contrast: while trajectories from D^2VINS diverge from the starting point, those estimated by D^2PGO consistently return to the start. Notably, in Fig. 11, D^2PGO accurately aligns the trajectories of two robots in the corridor (highlighted by the black box), unlike D^2VINS, where drift occurs due to the robots not traversing the corridor simultaneously. These results affirm the global consistency of D^2SLAM in far-field scenarios.

Fig. 1a displays the results of TSDF reconstruction using D^2PGO combined with submap fusion. The absence of noticeable drift in the map further also confirms the global consistency of D^2SLAM.

E. Scalability

In this section, we explore the scalability of computational performance.

1) Front-end: The computational demands of remote feature matching and loop closure detection in the front-end of D^2SLAM scale approximately linearly with the number of nearby robots. These algorithms are efficient, and their performance in real-world experiments is detailed in Table VI.

2) Back-end: The back-end dynamics are more complex. We adopt distributed optimization methods for graph optimization in this paper. In practice, D^2VINS implements a tailored landmark selection algorithm, MLS, balancing computational efficiency, accuracy, and robustness. This algorithm allows the number of visual measurements processed by D^2VINS to scale with the increase in robots, up to the τ_m limit. We have chosen a relatively high τ_m value to enhance the robustness of D^2VINS.

In Figs. 13 and 14, we demonstrate the scalability of D^2VINS and DPGO by charting the average computation time per operation against the increasing swarm size. Notably, on the TUM Room 5 dataset, there is a slight decrease in rotation accuracy with more robots, likely due to reduced inter-robot overlap in stereo camera FoVs. This issue doesn't occur in the OmniLongYaw 5 dataset with omnidirectional cameras. As the swarm size expands, the problem size grows quadratically. However, D^2SLAM's computational requirement for optimization increases at no more than a linear rate, showcasing the algorithm's efficiency. Furthermore, D^2SLAM provides flexibility in managing computational load through adjustable parameters τ_m and τ_l, suitable for diverse computational

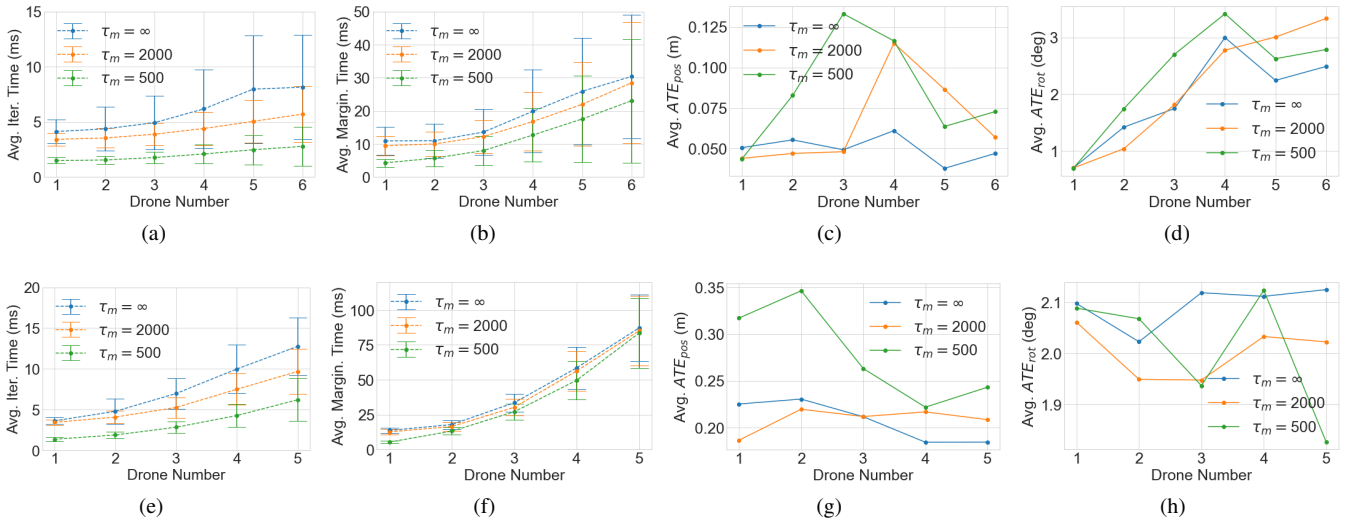


Fig. 13: The image illustrates the average time per iteration, marginalization time, translation error, and rotation error of D^2VINS for varying numbers of robots. We set $\tau_l = 200$ for the evaluation and compared results across different τ_m values. Sub-figures a)-d) represent stereo camera results derived from the TUM Room 5 dataset, while e)-h) show results from quad fisheye cameras using the OmniLongYaw 5 dataset.

TABLE VI: Typical computational time and running frequency of each module in the real-world experiment of D^2VINS.

Item	Image Preprocesing	Feature Tracking (Local)	Feature Tracking (Remote)	Optimization	Marginalization
Frequency	15 Hz	15 Hz	5 Hz	5 Hz	5 Hz
Time cost	41.7ms	2.67ms	1.64ms	56.23ms	16.88ms

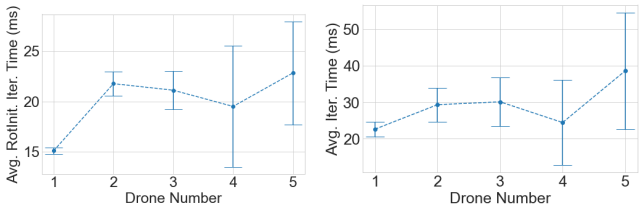


Fig. 14: The image depicts the average iteration time of D^2PGO with varying numbers of robots, derived from the TUM Corr 5 dataset. Sub-figure a) presents the results of rotation initialization, while e)-h) show the results of ARockPGO.

platforms. Reducing computational complexity through these parameters doesn't significantly compromise accuracy.

The time taken for marginalization increases with the number of robots, although, in theory, marginalization computation should be similar to a single optimization linear iteration. This discrepancy is primarily due to the ceres-solver framework's conventional approach of converting marginalization results into square root form for optimization [12], [60], which we plan to further optimize in the future.

F. Communication

Effective communication is pivotal for successful aerial swarm operations. To shed light on this aspect, Table VII details the sizes and broadcast frequencies of key messages transmitted by D^2SLAM on each UAV. Additionally, Table VIII presents a comparison between D^2SLAM's communication strategy (as detailed in Sect. IV-B) and a baseline method. This baseline strategy involves broadcasting all measurements (represented as complete keyframes in our context) without changing communication modes and is referenced in [1]. D^2SLAM's tailored strategy significantly reduces communication volume, especially when UAVs are widely dispersed.

Transitioning from the broader communication strategy, we delve into specifics for a 5-UAV swarm. Here, the maximum value of p^k reaches 51, leading to a maximum D^2VINS update size of 3.5kB per timestep. The value of e^k, which varies depending on the map and environment. For example, in TUM Corr 5 dataset, a typical e^k value is 1262, resulting in each D^2PGO update being approximately 81kB.

G. Real-world Experiments of D^2SLAM

Beyond dataset validation, we also test D^2SLAM in realistic environments using our UAV platforms. In these real-world experiments, as depicted in Fig. 15 and Table IX, D^2SLAM successfully handles limitations in computing power and communication, performing as anticipated. The table includes data from two flights, conducted by UAVs equipped with Realsense d435i cameras and utilizing the stereo version of D^2SLAM. In the *UAV2 NoYaw* experiment, the UAVs maintain fixed yaw angles, whereas in *UAV2 Yaw*, they rotate yaw angles according to their flight paths.

D^2SLAM demonstrates robust global consistency in both scenarios, with the ATE of D^2PGO being only about ten centimeters for flights ranging between 50 and 70 meters. This ATE accuracy of D^2PGO surpasses that of D^2VINS, highlighting the crucial role of D^2PGO in achieving global consistency for D^2SLAM. In terms of relative localization, D^2SLAM achieves centimeter-level accuracy with fixed yaw. However, the rotating yaw experiment reveals reduced relative

TABLE VII: Typical message sizes and broadcast frequencies in D^2SLAM. The terms 'D2VINS Update' and 'D2PGO Update' in the table refer to messages used for synchronization in swarm optimization within D^2SLAM. Here, p^k represents the number of poses in each UAV's sliding window, and e^k denotes the sum of dual states in the ARock algorithm.

Item	Complete Keyframe (Stereo)	Complete Keyframe (Omni)	Compact Keyframe (Stereo)	Compact Keyframe (Omni)	D^2VINS Update	D^2PGO Update
Size	20.1kB	56.3kB	1.2kB	4.8kB	$0.188 + 0.036p^k$kB	$0.056 + 0.064e^k$kB
Frequency	5	5	5	5	5	1

TABLE VIII: The total communication volume in MB of D^2SLAM in baseline mode and D^2SLAM mode.

Dataset	Mode	Front-end	Back-end	
			D^2VINS	D^2PGO
OmniLongYaw 5	Baseline	1546.7	75.9	2340.9
	D^2SLAM	984.2	70.8	2179.4
TUM ROOM 5	Baseline	266.4	1.6	42.0
	D^2SLAM	244.2	1.2	31.0
TUM Corr 5	Baseline	344.2	2.4	60.1
	D^2SLAM	323.0	2.9	74.1
HKUST RI 3	Baseline	231.8	10.1	13.4
	D^2SLAM	105.4	9.6	11.9

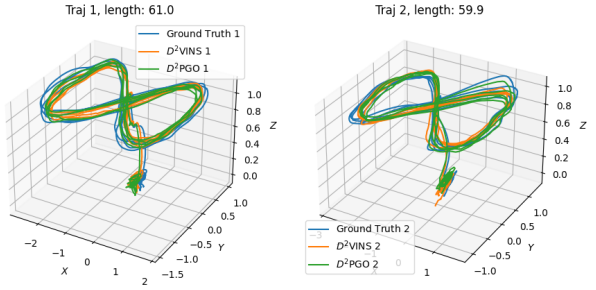


Fig. 15: The images display trajectories estimated by D^2VINS and D^2PGO in a real-world experiment.

localization accuracy, a consequence of the stereo version's limited FoV, as previously discussed.

Table VI displays the performance and operating frequency of key D^2SLAM algorithms when executed on the onboard computer during real-world experiments. Furthermore, communication between UAVs in these experiments is smooth: the average communication delay is only 23.89ms. This demonstrates that D^2SLAM meets real-time operational requirements in real-world aerial swarm systems.

X. Conclusion and Future Work

In this paper, we introduce D^2SLAM, a distributed and decentralized collaborative visual-inertial SLAM system. Our experiments demonstrate that D^2SLAM excels in real-time, high-precision local localization, and maintains high-precision relative localization when UAVs are in proximity, showcasing its capability for accurate near-field state estimation. Additionally, D^2SLAM can simultaneously estimate globally consistent trajectories, ensuring far-field state estimation with global consistency. With its flexible sensor configuration and versatile state estimation capabilities, D^2SLAM is poised to advance aerial swarm research, with applications ranging from self-assembling aerial swarms and cooperative transportation to inter-UAV collision avoidance and exploration of unknown environments.

While D^2SLAM, as presented in this paper, demonstrates considerable potential, it also has areas for future improvement: 1) Despite its distributed backend, the scalability of swarm size is currently constrained by communication and front-end computing capabilities. Future enhancements to D^2SLAM aim to enable its application to significantly larger aerial swarms. 2) Unlike our previous work, D^2SLAM adopts a more traditional visual SLAM approach without incorporating relative measurements such as UWB or mutual visual detection. This design choice makes D^2SLAM adaptable to a wider range of scenarios, including those with occlusions affecting UWB measurements or challenges in accurate visual identification of robots. However, recognizing the efficiency of relative measurements in certain contexts, we plan to integrate these into D^2SLAM in future developments. 3) Future improvement will also investigate leveraging hardware layer information or dense map data to assess communication quality between UAVs. This initiative aims to optimize network bandwidth utilization for state estimation algorithms.

Furthermore, we envision D^2SLAM's applicability extending beyond multi-robot systems to serve as a distributed visual-inertial SLAM system within a single robot. For example, deploying multiple D^2SLAM nodes on a single rigid UAV or ground robot could enhance state estimation accuracy and resilience against individual node failures. These nodes could also form distributed stereo camera systems to extend the range of environmental perception, as explored in [31]. Additionally, deploying D^2SLAM on multi-rigid robots, such as multi-legged platforms, and incorporating specific constraints could further refine their state and environmental estimations while offering redundancy.

References

[1] H. Xu, Y. Zhang, B. Zhou, L. Wang, X. Yao, G. Meng, and S. Shen, "Omni-Swarm: A Decentralized Omnidirectional Visual–Inertial–UWB State Estimation System for Aerial Swarms," *IEEE Trans. Robot. (TRO)*, 2022.

[2] Y. Tian, Y. Chang, F. H. Arias, C. Nieto-Granda, J. P. How, and L. Carlone, "Kimera-multi: robust, distributed, dense metric-semantic slam for multi-robot systems," *IEEE Transactions on Robotics*, 2022.

[3] P.-Y. Lajoie, B. Ramtoula, Y. Chang, L. Carlone, and G. Beltrame, "DOOR-SLAM: Distributed, online, and outlier resilient slam for robotic teams," *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 1656–1663, 2020.

[4] S. Choudhary, L. Carlone, C. Nieto, J. Rogers, H. I. Christensen, and F. Dellaert, "Distributed mapping with privacy and communication constraints: Lightweight algorithms and object-based models," *The International Journal of Robotics Research*, vol. 36, no. 12, pp. 1286–1311, 2017.

[5] X. Zhou, X. Wen, Z. Wang, Y. Gao, H. Li, Q. Wang, T. Yang, H. Lu, Y. Cao, C. Xu *et al.*, "Swarm of micro flying robots in the wild," *Science Robotics*, vol. 7, no. 66, p. eabm5954, 2022.

[6] B. Zhou, H. Xu, and S. Shen, "Racer: Rapid collaborative exploration with a decentralized multi-uav system," *arXiv preprint arXiv:2209.08533*, 2022.

TABLE IX: The table presents D^2SLAM's accuracy results from two flight experiments. The alignment approach for evaluating accuracy is consistent with that used in Table V.

Experiment	Avg. Traj. Len.	ATE_{pos}		ATE_{rot}		RE_{pos}			RE_{rot}
		D^2VINS	D^2PGO	D^2VINS	D^2PGO	x	y	z	
UAV2 NoYaw	51.0	0.103	0.090	2.24°	1.95°	0.047	0.067	0.056	3.36°
UAV2 Yaw	73.3	0.219	0.147	5.69°	4.17°	0.124	0.144	0.100	8.88°

[7] D. Saldana, B. Gabrich, G. Li, M. Yim, and V. Kumar, "Modquad: The flying modular structure that self-assembles in midair," in *2018 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2018, pp. 691–698.

[8] G. Loianno and V. Kumar, "Cooperative transportation using small quadrotors using monocular vision and inertial sensing," *IEEE Robotics and Automation Letters*, vol. 3, no. 2, pp. 680–687, 2017.

[9] X. Zhou, J. Zhu, H. Zhou, C. Xu, and F. Gao, "EGO-Swarm: A Fully Autonomous and Decentralized Quadrotor Swarm System in Cluttered Environments," *arXiv preprint arXiv:2011.04183*, 2020.

[10] P. C. Lusk, X. Cai, S. Wadhwania, A. Paris, K. Fathian, and J. P. How, "A Distributed Pipeline for Scalable, Deconflicted Formation Flying," *IEEE Robotics and Automation Letters*, vol. 5, no. 4, pp. 5213–5220, 2020.

[11] H. Xu, L. Wang, Y. Zhang, K. Qiu, and S. Shen, "Decentralized visual-inertial-UWB fusion for relative state estimation of aerial swarm," in *Proc. of the IEEE Intl. Conf. on Robot. and Autom. (ICRA)*. IEEE, 2020, pp. 8776–8782.

[12] T. Qin, P. Li, and S. Shen, "Vins-mono: A robust and versatile monocular visual-inertial state estimator," *IEEE Transactions on Robotics*, vol. 34, no. 4, pp. 1004–1020, 2018.

[13] W. Shi, Q. Ling, K. Yuan, G. Wu, and W. Yin, "On the linear convergence of the admm in decentralized consensus optimization," *IEEE Transactions on Signal Processing*, vol. 62, no. 7, pp. 1750–1761, 2014.

[14] R. Zhang, S. Zhu, T. Fang, and L. Quan, "Distributed very large scale bundle adjustment by global camera consensus," in *Proceedings of the IEEE International Conference on Computer Vision*, 2017, pp. 29–38.

[15] A. Eriksson, J. Bastian, T.-J. Chin, and M. Isaksson, "A consensus-based framework for distributed bundle adjustment," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 1754–1762.

[16] Z. Peng, Y. Xu, M. Yan, and W. Yin, "Arock: an algorithmic framework for asynchronous parallel coordinate updates," *SIAM Journal on Scientific Computing*, vol. 38, no. 5, pp. A2851–A2879, 2016.

[17] D. Rosen, L. Carlone, A. Bandeira, and J. Leonard, "SE-Sync: A certifiably correct algorithm for synchronization over the special Euclidean group," *Intl. J. of Robotics Research*, vol. 38, no. 2–3, pp. 95–125, Mar. 2019.

[18] J. Briales and J. Gonzalez-Jimenez, "Cartan-sync: Fast and global se (d)-synchronization," *IEEE Robotics and Automation Letters*, vol. 2, no. 4, pp. 2127–2134, 2017.

[19] F. Dellaert, M. Kaess *et al.*, "Factor Graphs for Robot Perception," *Foundations and Trends® in Robotics*, vol. 6, no. 1-2, pp. 1–139, 2017.

[20] R. Mur-Artal, J. M. M. Montiel, and J. D. Tardos, "ORB-SLAM: a versatile and accurate monocular slam system," *IEEE transactions on robotics*, vol. 31, no. 5, pp. 1147–1163, 2015.

[21] V. Reijgwart, A. Millane, H. Oleynikova, R. Siegwart, C. Cadena, and J. Nieto, "Voxgraph: Globally consistent, volumetric mapping using signed distance function submaps," *IEEE Robotics and Automation Letters*, 2020.

[22] A. Cunningham, M. Paluri, and F. Dellaert, "DDF-SAM: Fully distributed SLAM using constrained factor graphs," in *Proc. of the IEEE/RSJ Intl. Conf. on Intell. Robots and Syst.(IROS)*. IEEE, 2010, pp. 3025–3030.

[23] A. Cunningham, V. Indelman, and F. Dellaert, "DDF-SAM 2.0: Consistent distributed smoothing and mapping," in *Proc. of the IEEE Intl. Conf. on Robot. and Autom. (ICRA)*. IEEE, 2013, pp. 5220–5227.

[24] N. Michael, S. Shen, K. Mohta, V. Kumar, K. Nagatani, Y. Okada, S. Kiribayashi, K. Otake, K. Yoshida, K. Ohno *et al.*, "Collaborative mapping of an earthquake damaged building via ground and aerial robots," in *Field and Service Robotics*. Springer, 2014, pp. 33–47.

[25] D. P. Bertsekas and J. N. Tsitsiklis, *Parallel and Distributed Computation: Numerical Methods*. Prentice-Hall, Inc., 1989.

[26] Y. Tian, A. Koppel, A. S. Bedi, and J. P. How, "Asynchronous and parallel distributed pose graph optimization," *IEEE Robotics and Automation Letters*, vol. 5, no. 4, pp. 5819–5826, 2020.

[27] N. Boumal, "An introduction to optimization on smooth manifolds," [Online] Available:, Nov 2020. [Online]. Available: http://www.nicolasboumal.net/book

[28] Y. Tian, K. Khosoussi, D. M. Rosen, and J. P. How, "Distributed certifiably correct pose-graph optimization," *IEEE Trans. Robot. (TRO)*, 2021.

[29] A. Rosinol, M. Abate, Y. Chang, and L. Carlone, "Kimera: an Open-Source Library for Real-Time Metric-Semantic Localization and Mapping," in *2020 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2020, pp. 1689–1696.

[30] B. Triggs, P. F. McLauchlan, R. I. Hartley, and A. W. Fitzgibbon, "Bundle adjustment—a modern synthesis," in *International workshop on vision algorithms*. Springer, 1999, pp. 298–372.

[31] M. Karrer and M. Chli, "Distributed variable-baseline stereo SLAM from two UAVs," in *2021 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2021, pp. 82–88.

[32] P. Zhu, P. Geneva, W. Ren, and G. Huang, "Distributed Visual-Inertial Cooperative Localization," *Proc. of the IEEE/RSJ Intl. Conf. on Intell. Robots and Syst.(IROS)*, 2021.

[33] P. Zhu, Y. Yang, W. Ren, and G. Huang, "Cooperative Visual-Inertial Odometry," in *Proc. of the IEEE Intl. Conf. on Robot. and Autom. (ICRA)*, 2021.

[34] R. Jung and S. Weiss, "Scalable recursive distributed collaborative state estimation for aided inertial navigation," in *2021 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2021, pp. 1896–1902.

[35] K. Guo, X. Li, and L. Xie, "Ultra-wideband and odometry-based cooperative relative localization with application to multi-uav formation control," *IEEE Transactions on Cybernetics*, 2019.

[36] T. Ziegler, M. Karrer, P. Schmuck, and M. Chli, "Distributed Formation Estimation via Pairwise Distance Measurements," *IEEE Robotics and Automation Letters*, 2021.

[37] T. H. Nguyen, T.-M. Nguyen, and L. Xie, "Range-Focused Fusion of Camera-IMU-UWB for Accurate and Drift-Reduced Localization," *IEEE Robotics and Automation Letters*, vol. 6, no. 2, pp. 1678–1685, 2021.

[38] T. Nguyen, K. Mohta, C. J. Taylor, and V. Kumar, "Vision-based Multi-MAV Localization with Anonymous Relative Measurements Using Coupled Probabilistic Data Association Filter," 2020.

[39] G. Huang, "Visual-inertial navigation: A concise review," in *2019 international conference on robotics and automation (ICRA)*. IEEE, 2019, pp. 9572–9582.

[40] T. Qin, P. Li, and S. Shen, "Relocalization, global optimization and map merging for monocular visual-inertial slam," in *2018 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2018, pp. 1197–1204.

[41] K. Qiu, T. Liu, and S. Shen, "Model-based global localization for aerial robots using edge alignment," *IEEE Robotics and Automation Letters*, vol. 2, no. 3, pp. 1256–1263, 2017.

[42] Z. Zhang and D. Scaramuzza, "A Tutorial on Quantitative Trajectory Evaluation for Visual(-Inertial) Odometry," in *IEEE/RSJ Int. Conf. Intell. Robot. Syst. (IROS)*, 2018.

[43] R. Zhang and J. Kwok, "Asynchronous distributed admm for consensus optimization," in *International conference on machine learning*. PMLR, 2014, pp. 1701–1709.

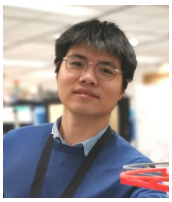
[44] M. Van Steen and A. Tanenbaum, "Distributed systems principles and paradigms," *Network*, vol. 2, p. 28, 2002.

[45] W. Gao and S. Shen, "Dual-fisheye omnidirectional stereo," in *Proc. of the IEEE/RSJ Intl. Conf. on Intell. Robots and Syst.(IROS)*, 2017, pp. 6715–6722.

[46] C. Won, H. Seok, Z. Cui, M. Pollefeys, and J. Lim, "OmniSLAM: Omnidirectional localization and dense mapping for wide-baseline multi-camera systems," in *2020 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2020, pp. 559–566.

[47] E. Plaut, E. Ben Yaacov, and B. El Shlomo, "3d object detection from a single fisheye image without a single fisheye training image," in *Proceedings of the IEEE/CVF Conference on Computer Vision and*

Pattern Recognition, 2021, pp. 3659–3667.
[48] P.-E. Sarlin, C. Cadena, R. Siegwart, and M. Dymczyk, "From coarse to fine: Robust hierarchical localization at large scale," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2019, pp. 12 716–12 725.
[49] D. DeTone, T. Malisiewicz, and A. Rabinovich, "Superpoint: Self-supervised interest point detection and description," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition Workshops*, 2018, pp. 224–236.
[50] R. Mur-Artal and J. D. Tardós, "ORB-SLAM2: an Open-Source SLAM System for Monocular, Stereo and RGB-D Cameras," *IEEE Transactions on Robotics*, vol. 33, no. 5, pp. 1255–1262, 2017.
[51] D. G. Lowe, "Distinctive image features from scale-invariant keypoints," *International journal of computer vision*, vol. 60, pp. 91–110, 2004.
[52] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," *IEEE Transactions on Big Data*, 2019.
[53] L. Kneip, H. Li, and Y. Seo, "Upnp: An optimal o (n) solution to the absolute pose problem with universal applicability," in *European conference on computer vision*. Springer, 2014, pp. 127–142.
[54] L. Kneip and P. Furgale, "Opengv: A unified and generalized approach to real-time calibrated geometric vision," in *2014 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2014, pp. 1–8.
[55] J. Civera, A. J. Davison, and J. M. Montiel, "Inverse depth parametrization for monocular slam," *IEEE transactions on robotics*, vol. 24, no. 5, pp. 932–945, 2008.
[56] P. J. Huber, "Robust estimation of a location parameter," in *Breakthroughs in Statistics*. Springer, 1992, pp. 492–518.
[57] J. L. Blanco-Claraco, "A tutorial on se(3) transformation parameterizations and on-manifold optimization," *arXiv preprint arXiv:2103.15980*, 2021.
[58] F. L. Markley, Y. Cheng, J. L. Crassidis, and Y. Oshman, "Averaging quaternions," *Journal of Guidance, Control, and Dynamics*, vol. 30, no. 4, pp. 1193–1197, 2007.
[59] H. Xu and S. Shen, "BDPGO: balanced distributed pose graph optimization framework for swarm robotics," *arXiv preprint arXiv:2109.04502*, 2021.
[60] S. Leutenegger, S. Lynen, M. Bosse, R. Siegwart, and P. Furgale, "Keyframe-based visual–inertial odometry using nonlinear optimization," *The International Journal of Robotics Research*, vol. 34, no. 3, pp. 314–334, 2015.
[61] S. Agarwal, K. Mierle, and Others, "Ceres solver," http://ceres-solver.org.
[62] K. Madsen, H. B. Nielsen, and O. Tingleff, "Methods for non-linear least squares problems," 2004.
[63] G. Grisetti, R. Kümmerle, C. Stachniss, and W. Burgard, "A tutorial on graph-based slam," *IEEE Intelligent Transportation Systems Magazine*, vol. 2, no. 4, pp. 31–43, 2010.
[64] L. Carlone, R. Tron, K. Daniilidis, and F. Dellaert, "Initialization techniques for 3d slam: a survey on rotation estimation and its use in pose graph optimization," in *2015 IEEE international conference on robotics and automation (ICRA)*. IEEE, 2015, pp. 4597–4604.
[65] R. Hartley, J. Trumpf, Y. Dai, and H. Li, "Rotation averaging," *International journal of computer vision*, vol. 103, no. 3, pp. 267–305, 2013.
[66] D. Martinec and T. Pajdla, "Robust rotation and translation estimation in multiview reconstruction," in *2007 IEEE Conference on Computer Vision and Pattern Recognition*. IEEE, 2007, pp. 1–8.
[67] J. Nocedal and S. J. Wright, *Numerical optimization*. Springer, 1999.
[68] J. Sola, "Quaternion kinematics for the error-state kalman filter," *arXiv preprint arXiv:1711.02508*, 2017.
[69] J. J. Moré, "The levenberg-marquardt algorithm: implementation and theory," in *Numerical analysis*. Springer, 1978, pp. 105–116.
[70] J. G. Mangelson, D. Dominic, R. M. Eustice, and R. Vasudevan, "Pairwise consistent measurement set maximization for robust multi-robot map merging," in *2018 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2018, pp. 2916–2923.
[71] H. Yang, P. Antonante, V. Tzoumas, and L. Carlone, "Graduated non-convexity for robust spatial perception: From non-minimal solvers to global outlier rejection," *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 1127–1134, 2020.
[72] G. Guennebaud, B. Jacob *et al.*, "Eigen v3," http://eigen.tuxfamily.org, 2010.
[73] D. Schubert, T. Goll, N. Demmel, V. Usenko, J. Stückler, and D. Cremers, "The tum vi benchmark for evaluating visual-inertial odometry," in *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2018, pp. 1680–1687.
[74] L. Meier, D. Honegger, and M. Pollefeys, "Px4: A node-based multithreaded open source robotics framework for deeply embedded platforms," in *2015 IEEE international conference on robotics and automation (ICRA)*. IEEE, 2015, pp. 6235–6240.
[75] O. R. developers, "Onnx runtime," https://onnxruntime.ai/, 2021, version: x.y.z.
[76] Y. Hu, T.-M. Li, L. Anderson, J. Ragan-Kelley, and F. Durand, "Taichi: a language for high-performance computation on spatially sparse data structures," *ACM Transactions on Graphics (TOG)*, vol. 38, no. 6, pp. 1–16, 2019.
[77] K. M. Wurm, A. Hornung, M. Bennewitz, C. Stachniss, and W. Burgard, "Octomap: A probabilistic, flexible, and compact 3d map representation for robotic systems," in *Proc. of the IEEE Intl. Conf. on Robot. and Autom. (ICRA)*, vol. 2, Anchorage, AK, US, May 2010.
[78] H. Oleynikova, Z. Taylor, M. Fehr, R. Siegwart, and J. Nieto, "Voxblox: Incremental 3d euclidean signed distance fields for on-board mav planning," in *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2017, pp. 1366–1373.
[79] V. Reijgwart, A. Millane, H. Oleynikova, R. Siegwart, C. Cadena, and J. Nieto, "Voxgraph: Globally consistent, volumetric mapping using signed distance function submaps," *IEEE Robotics and Automation Letters*, vol. 5, no. 1, pp. 227–234, 2019.



Hao Xu received the B.Sc. degree in Physics from the University of Science and Technology, and the Ph.D. degree in Electronic and Computer Engineering (ECE) from the Hong Kong University of Science and Technology, Hong Kong, in 2023. He is currently working as an algorithm engineer at Shenzhen DJI Sciences and Technologies Ltd. His research interests include unmanned aerial vehicles, aerial swarm, state estimation, sensor fusion, localization, and mapping.



Peize LIU received the B.Sc. degree in Software Engineering from the University of Electronic Science and Technology of China, Chengdu, China, in 2022. He is currently working toward the Ph.D. degree with the Hong Kong University of Science and Technology, Hong Kong, under the supervision of Prof. Shaojie Shen. His research interests include unmanned aerial vehicles, state estimation in aerial swarm, and swarm system.



Xinyi Chen received the B.Sc. degree in Mathematics, and in Computer Science from the Hong Kong University of Science and Technology, Hong Kong, in 2021. She is currently working towards the Ph.D. degree with the Hong Kong University of Science and Technology, Hong Kong, under the supervision of Prof. Shaojie Shen. Her research interests include aerial vehicles, motion planning, autonomous exploration, aerial swarm, localization and mapping.



Shaojie Shen received his B.Eng. degree in Electronic Engineering from the Hong Kong University of Science and Technology (HKUST) in 2009. He received his M.S. in Robotics and Ph.D. in Electrical and Systems Engineering in 2011 and 2014, respectively, all from the University of Pennsylvania. He joined the Department of Electronic and Computer Engineering at the HKUST in September 2014 as an Assistant Professor, and is promoted to Associate Professor in July 2020. He is the founding director of the HKUST-DJI Joint Innovation Laboratory (HDJI Lab). His research interests are in the areas of robotics and unmanned aerial vehicles, with focus on state estimation, sensor fusion, localization and mapping, and autonomous navigation in complex environments.